phyCORE-i.MX 8M Plus BSP Manual PD24.1.1

PHYTEC Messtechnik GmbH

Oct 13, 2026

CONTENTS

1 1 PHYTEC Documentation **3**
1.1 1.1 Supported Hardware . . . 3

2 2 Getting Started **5**
2.1 2.1 Get the Image . . . 5
2.2 2.2 Write the Image to SD Card . . . 5
2.3 2.3 First Start-up . . . 8

3 3 Building the BSP **9**
3.1 3.1 Basic Set-Up . . . 9
3.2 3.2 Get the BSP . . . 9

4 4 Installing the OS **13**
4.1 4.1 Bootmode Switch (S3) . . . 13
4.2 4.2 Flash eMMC . . . 13
4.3 4.3 RAUC . . . 19

5 5 Development **21**
5.1 5.1 Host Network Preparation . . . 21
5.2 5.2 Booting the Kernel from a Network . . . 23
5.3 5.3 Working with UUU-Tool . . . 24
5.4 5.4 Standalone Build preparation . . . 26
5.5 5.5 U-Boot standalone build . . . 27
5.6 5.6 Kernel standalone build . . . 28
5.7 5.7 Accessing the Development states . . . 29
5.8 5.8 Accessing the Latest Upstream Support . . . 30
5.9 5.9 Format SD-Card . . . 30

6 6 Device Tree (DT) **37**
6.1 6.1 Introduction . . . 37
6.2 6.2 PHYTEC i.MX 8M Plus BSP Device Tree Concept . . . 37

7 7 Accessing Peripherals **39**
7.1 7.1 i.MX 8M Plus Pin Muxing . . . 39
7.2 7.2 RS232/RS485 . . . 40
7.3 7.3 Ethernet . . . 41
7.4 7.4 SD card . . . 42
7.5 7.5 e.MMC Devices . . . 43
7.6 7.6 SPI Master . . . 50
7.7 7.7 GPIOs . . . 50
7.8 7.8 LEDs . . . 53

7.9 7.9 I²C Bus . . . 53
7.10 7.10 EEPROM . . . 53
7.11 7.11 RTC . . . 54
7.12 7.12 USB Host Controller . . . 56
7.13 7.13 CAN FD . . . 57
7.14 7.14 Video . . . 59
7.15 7.15 Display . . . 59
7.16 7.16 Power Management . . . 60
7.17 7.17 Thermal Management . . . 62
7.18 7.18 Watchdog . . . 63
7.19 7.19 snvs Power Key . . . 64
7.20 7.20 On-Chip OTP Controller (OCOTP_CTRL) - eFuses . . . 64

The table below shows the Compatible BSPs for this manual:

Compatible BSPs	BSP Release Type	Yocto Version	BSP Release Date	BSP Status
BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1	Minor	Scarthgap	2024/04/08	Released

This BSP manual guides you through the installation and creation steps for the Board Support Package (BSP) and describes how to handle the interfaces for the **phyCORE-i.MX8M Plus Kit**. Furthermore, this document describes how to create BSP images from the source code. This is useful for those who need to change the default image and need a way to implement these changes in a simple and reproducible way. Further, some sections of this manual require executing commands on a personal computer (host). Any and all of these commands are assumed to be executed on a Linux Operating System.

Note

This document contains code examples that describe the communication with the board over the serial shell. The code examples lines begin with `host:~$`, `target:~$` or `u-boot=>`. This describes where the commands are to be executed. Only after these keywords must the actual command be copied.

CHAPTER

ONE

1 PHYTEC DOCUMENTATION

PHYTEC provides a variety of hardware and software documentation for all of its products. This includes any or all of the following:

QS Guide
A short guide on how to set up and boot a PHYTEC System-on-Module based board.

Hardware Manual
A detailed description of the System-on-Module and accompanying carrierboard.

Yocto Guide
A comprehensive guide for the Yocto version the phyCORE uses. This guide contains an overview of Yocto; introducing, installing, and customizing the PHYTEC BSP; how to work with programs like Poky and Bitbake; and much more.

BSP Manual
A manual specific to the BSP version of the phyCORE. Information such as how to build the BSP, booting, updating software, device tree, and accessing peripherals can be found here.

Development Environment Guide
This guide shows how to work with the Virtual Machine (VM) Host PHYTEC has developed and prepared to run various Development Environments. There are detailed step-by-step instructions for Eclipse and Qt Creator, which are included in the VM. There are instructions for running demo projects for these programs on a phyCORE product as well. Information on how to build a Linux host PC yourself is also a part of this guide.

Pin Muxing Table
phyCORE SOMs have an accompanying pin table (in Excel format). This table will show the complete default signal path, from the processor to the carrier board. The default device tree muxing option will also be included. This gives a developer all the information needed in one location to make muxing changes and design options when developing a specialized carrier board or adapting a PHYTEC phyCORE SOM to an application.

On top of these standard manuals and guides, PHYTEC will also provide Product Change Notifications, Application Notes, and Technical Notes. These will be done on a case-by-case basis. Most of the documentation can be found on the https://www.phytec.de/produkte/system-on-modules/phycore-imx-8m-plus/#downloads of our product.

1.1 1.1 Supported Hardware

On our web page, you can see all supported Machines with the available Article Numbers for this release: BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1 download.

If you choose a specific **Machine Name** in the section **Supported Machines**, you can see which **Article Numbers** are available under this machine and also a short description of the hardware information. In case

you only have the **Article Number** of your hardware, you can leave the **Machine Name** drop-down menu empty and only choose your **Article Number**. Now it should show you the necessary **Machine Name** for your specific hardware

1.1.1 1.1.1 phyBOARD-Pollux Components

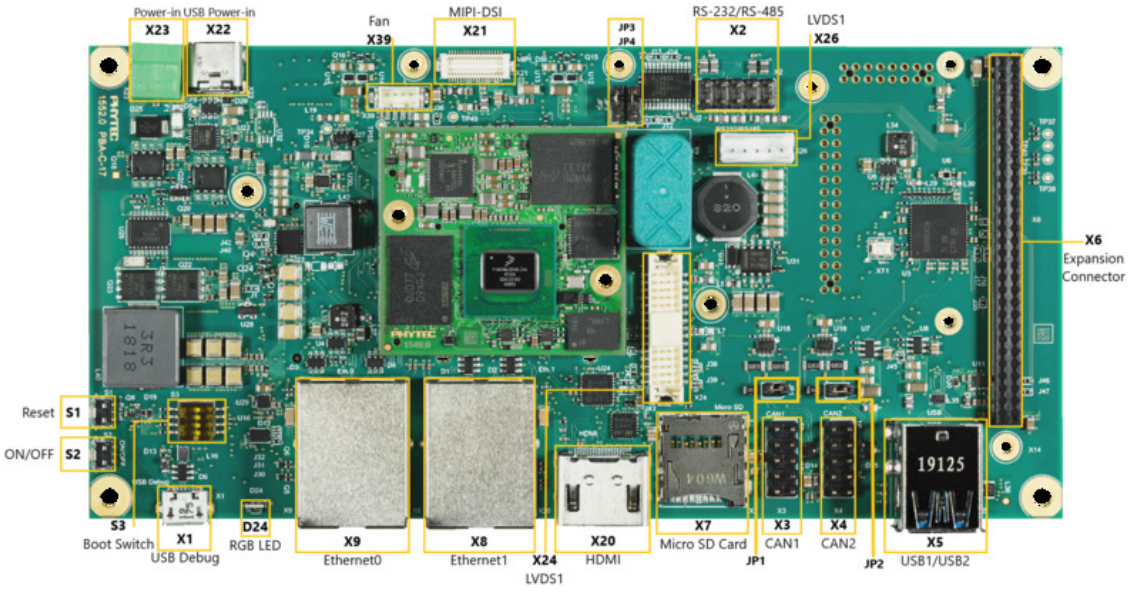


Fig. 1: **phyBOARD-Pollux Components (top)**

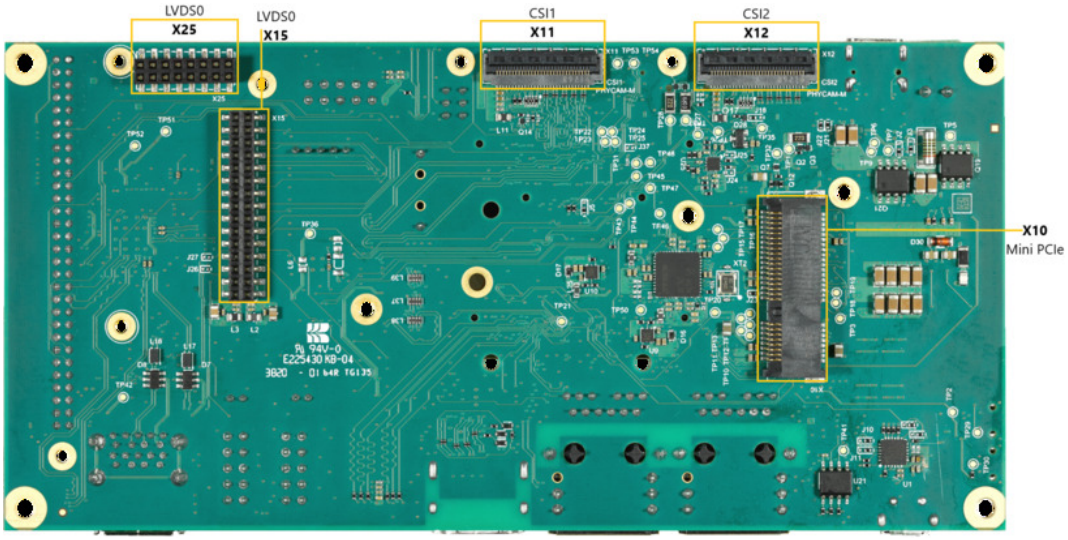


Fig. 2: **phyBOARD-Pollux Components (bottom)**

CHAPTER

TWO

2 GETTING STARTED

The **phyCORE-i.MX8M Plus Kit** is shipped with a pre-flashed SD card. It contains the phytec-qt6demo-image and can be used directly as a boot source. The eMMC is programmed with only a U-Boot by default. You can get all sources from the PHYTEC download server. This chapter explains how to flash a BSP image to SD card and how to start the board.

There are several ways to flash an image to SD card or even eMMC. Most notably using simple, sequential writing with the Linux command line tool `dd`. An alternative way is to use PHYTEC's system initialization program called partup, which makes it especially easy to format more complex systems. You can get prebuilt Linux binaries of partup from its release page. Also read partup's README for installation instructions.

2.1 2.1 Get the Image

The image contains all necessary files and makes sure partitions and any raw data are correctly written. Both the partup package and the WIC image, which can be flashed using `dd`, can be downloaded from the PHYTEC download server.

Get either the partup package or the WIC image from the download server:

```
host:~$ wget https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX8MP/BSP-Yocto-Ampliphy-i.
↪MX8MP-PD24.1.1/images/ampliphy-xwayland/phyboard-pollux-imx8mp-3/phytec-qt6demo-image-phyboard-
↪pollux-imx8mp-3.rootfs.partup
host:~$ wget https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX8MP/BSP-Yocto-Ampliphy-i.
↪MX8MP-PD24.1.1/images/ampliphy-xwayland/phyboard-pollux-imx8mp-3/phytec-qt6demo-image-phyboard-
↪pollux-imx8mp-3.rootfs.wic
```

Note

For eMMC, more complex partitioning schemes or even just large images, we recommend using the partup package, as it is faster in writing than `dd` and allows for a more flexible configuration of the target flash device.

2.2 2.2 Write the Image to SD Card

Warning

To create your bootable SD card, you must have root privileges on your Linux host PC. Be very careful when specifying the destination device! All files on the selected device will be erased immediately without any further query!

Selecting the wrong device may result in **data loss** and e.g. could erase your currently running system on your host PC!

2.2.1 2.2.1 Finding the Correct Device

To create your bootable SD card, you must first find the correct device name of your SD card and possible partitions. If any partitions of the SD cards are mounted, unmount those before you start copying the image to the SD card.

1. In order to get the correct device name, remove your SD card and execute:

```
host:~$ lsblk
```

2. Now insert your SD card and execute the command again:

```
host:~$ lsblk
```

3. Compare the two outputs to find the new device names listed in the second output. These are the device names of the SD card (device and partitions if the SD card was formatted).
4. In order to verify the device names being found, execute the command `sudo dmesg`. Within the last lines of its output, you should also find the device names, e.g. `/dev/sde` or `/dev/mmcblk0` (depending on your system).

Alternatively, you may use a graphical program of your choice, like GNOME Disks or KDE Partition Manager, to find the correct device.

Now that you have the correct device name, e.g. `/dev/sde`, you can see the partitions which must be unmounted if the SD card is formatted. In this case, you will also find the device name with an appended number (e.g. `/dev/sde1`) in the output. These represent the partitions. Some Linux distributions automatically mount partitions when the device gets plugged in. Before writing, however, these need to be unmounted to avoid data corruption.

Unmount all those partitions, e.g.:

```
host:~$ sudo umount /dev/sde1
host:~$ sudo umount /dev/sde2
```

Now, the SD card is ready to be flashed with an image, using either `partup`, `dd` or `bmap-tools`.

2.2.2 2.2.2 Using bmap-tools

One way to prepare an SD card is using bmap-tools. Yocto automatically creates a block map file (`<IMAGENAME>-<MACHINE>.wic.bmap`) for the WIC image that describes the image content and includes checksums for data integrity. *bmaptool* is packaged by various Linux distributions. For Debian-based systems install it by issuing:

```
host:~$ sudo apt install bmap-tools
```

Flash a WIC image to SD card by calling:

```
host:~$ bmaptool copy phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic /dev/<your_device>
```

Replace <your_device> with your actual SD card's device name found previously, and make sure to place the file `<IMAGENAME>-<MACHINE>.wic.bmap` alongside the regular WIC image file, so bmaptool knows which blocks to write and which to skip.

Warning

bmaptool only overwrites the areas of an SD card where image data is located. This means that a previously written U-Boot environment may still be available after writing the image.

2.2.3 2.2.3 Using partup

Writing to an SD card with partup is done in a single command:

```
host:~$ sudo partup install phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.partup /dev/
↪<your_device>
```

Make sure to replace <your_device> with your actual device name found previously.

Further usage of partup is explained at its official documentation website.

Warning

Host systems which are using resize2fs version 1.46.6 and older (e.g. Ubuntu 22.04) are not able to write partup packages created with Yocto Mickledore or newer to SD-Card. This is due to a new default option in resize2fs which causes an incompatibility. See release notes.

Note

partup has the advantage of allowing to clear specific raw areas in the MMC user area, which is used in our provided partup packages to erase any existing U-Boot environments. This is a known issue *bmaptool* does not solve, as mentioned in the previous chapter.

Another key advantage of partup over other flashing tools is that it allows configuring MMC specific parts, like writing to eMMC boot partitions, without the need to call multiple other commands when writing.

2.2.4 2.2.4 Using `dd`

After having unmounted all SD card's partitions, you can create your bootable SD card.

Some PHYTEC BSPs produce uncompressed images (with filename-extension *.wic), and some others produce compressed images (with filename-extension *.wic.xz).

To flash an uncompressed images (*.wic) use command below:

```
host:~$ sudo dd if=phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic of=/dev/<your_device>
↪ bs=1M conv=fsync status=progress
```

Or to flash a compressed images (*.wic.xz) use that command:

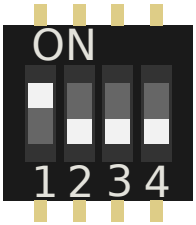
```
host:~$ xzcat phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic.xz | sudo dd of=/dev/
↪<your_device> bs=1M conv=fsync status=progress
```

Again, make sure to replace <your_device> with your actual device name found previously.

The parameter `conv=fsync` forces a sync operation on the device before `dd` returns. This ensures that all blocks are written to the SD card and none are left in memory. The parameter `status=progress` will print out information on how much data is and still has to be copied until it is finished.

2.3 2.3 First Start-up

- To boot from an SD card, the *bootmode switch (S3)* needs to be set to the following position:



- Insert the SD card
- Connect the target and the host with **micro USB** on *(X1)* debug USB
- Power up the board

CHAPTER

THREE

3 BUILDING THE BSP

This section will guide you through the general build process of the i.MX 8M Plus BSP using Yocto and the phyLinux script. For more information about our meta-layer or Yocto in general visit: Yocto Reference Manual (scarthgap).

3.1 3.1 Basic Set-Up

If you have never created a Phytec BSP with Yocto on your computer, you should take a closer look at the chapter BSP Workspace Installation in the Yocto Reference Manual (scarthgap).

3.2 3.2 Get the BSP

There are two ways to get the BSP sources. You can download the complete BSP sources from our BSP downloads page; or you can fetch and build it yourself with Yocto. This is particularly useful if you want to make customizations.

The phyLinux script is a basic management tool for PHYTEC Yocto BSP releases written in Python. It is mainly a helper to get started with the BSP sources structure.

- Create a fresh project folder, get phyLinux, and make the script executable:

```
host:~$ mkdir ~/yocto
host:~$ cd yocto/
host:~/yocto$ wget https://download.phytec.de/Software/Linux/Yocto/Tools/phyLinux
host:~/yocto$ chmod +x phyLinux
```

> **Warning**
>
> A clean folder is important because phyLinux will clean its working directory. Calling phyLinux from a directory that isn't empty will result in a warning.

- Run phyLinux:

```
host:~/yocto$ ./phyLinux init
```

> **Note**
>
> On the first initialization, the phyLinux script will ask you to install the Repo tool in your `/usr/local/bin` directory.

- During the execution of the init command, you need to choose your processor platform (SoC), PHYTEC's BSP release number, and the hardware (MACHINE) you are working on.

> **Note**
>
> If you cannot identify your board with the information given in the selector, have a look at the invoice for the product. And have a look at the webpage of our BSP.

- It is also possible to pass this information directly using command line parameters:

```
host:~/yocto$ DISTRO=ampliphy-xwayland MACHINE=phyboard-pollux-imx8mp-3 ./phyLinux init -p␣
↪imx8mp -r BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1
```

After the execution of the init command, phyLinux will print a few important notes. For example, it will print your git identity, SOC and BSP release which was selected as well as information for the next steps in the build process.

3.2.1 3.2.1 Starting the Build Process

- Set up the shell environment variables:

```
host:~/yocto$ source sources/poky/oe-init-build-env
```

> **Note**
>
> This needs to be done every time you open a new shell for starting builds.

- The current working directory of the shell should change to build/.
- Open the main configuration file and accept the GPU and VPU binary license agreements. Do this by uncommenting the corresponding line, as below.

```
host:~/yocto/build$ vim conf/local.conf
# Uncomment to accept NXP EULA
# EULA can be found under ../sources/meta-freescale/EULA
ACCEPT_FSL_EULA = "1"
```

- Build your image:

```
host:~/yocto/build$ bitbake phytec-qt6demo-image
```

> **Note**
>
> For the first build we suggest starting with our smaller non-graphical image phytec-headless-image to see if everything is working correctly.
>
> ```
> host:~/yocto/build$ bitbake phytec-headless-image
> ```
>
> The first compile process takes about 40 minutes on a modern Intel Core i7. All subsequent builds will use the filled caches and should take about 3 minutes.

3.2.2 3.2.2 BSP Images

All images generated by Bitbake are deployed to `~/yocto/build/deploy*/images/<machine>`. The following list shows for example all files generated for the phyboard-pollux-imx8mp-3 machine:

- **u-boot.bin**: Binary compiled U-boot bootloader (U-Boot). Not the final Bootloader image!
- **oftree**: Default kernel device tree
- **u-boot-spl.bin**: Secondary program loader (SPL)
- **bl31-imx8mp.bin**: ARM Trusted Firmware binary
- **lpddr4_pmu_train_1d_dmem_202006.bin, lpddr4_pmu_train_1d_imem_202006.bin, lpddr4_pmu_train_2d_dmem_202006.bin, lpddr4_pmu_train_2d_imem_202006.bin**: DDR PHY firmware images
- **Image**: Linux kernel image
- **Image.config**: Kernel configuration
- **imx8mp-phyboard-pollux-rdk*.dtb**: Kernel device tree file
- **phytec-qt6demo-image*.tar.gz**: Root file system
- **phytec-qt6demo-image*.wic**: SD card image

CHAPTER

FOUR

4 INSTALLING THE OS

4.1 4.1 Bootmode Switch (S3)

Tip

Hardware revision baseboard: 1552.2

The phyBOARD-Pollux features a boot switch with four individually switchable ports to select the phyCORE-i.MX 8M Plus default bootsource.

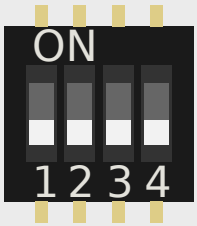


Fig. 1: eMMC

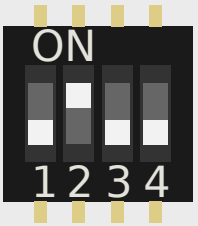


Fig. 2: Internal Fuses

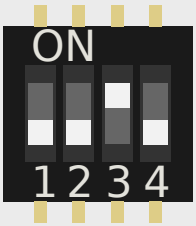


Fig. 3: SPI NOR

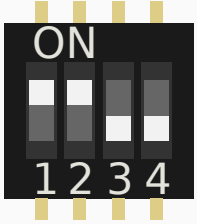


Fig. 4: USB Serial Download

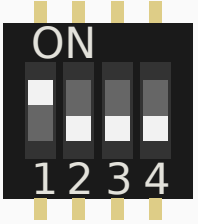


Fig. 5: SD Card

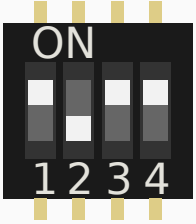


Fig. 6: Test Mode

4.2 4.2 Flash eMMC

To boot from eMMC, make sure that the BSP image is flashed correctly to the eMMC and the *bootmode switch (S3)* is set to **eMMC**.

Warning

When eMMC and SD card are flashed with the same (identical) image, the UUIDs of the boot partitions are also identical. If the SD card is connected when booting, this leads to non-deterministic behavior as

Linux mounts the boot partition based on UUID.

```
target:~$ blkid
```

can be run to inspect whether the current setup is affected. If `mmcblk2p1` and `mmcblk1p1` have an identical UUID, the setup is affected.

4.2.1 4.2.1 Flash eMMC from Network

i.MX 8M Plus boards have an Ethernet connector and can be updated over a network. Be sure to set up the development host correctly. The IP needs to be set to 192.168.3.10, the netmask to 255.255.255.0, and a TFTP server needs to be available. From a high-level point of view, an eMMC device is like an SD card. Therefore, it is possible to flash the **WIC image** (`<name>.wic`) from the Yocto build system directly to the eMMC. The image contains the bootloader, kernel, device tree, device tree overlays, and root file system.

4.2.1.1 Flash eMMC from Network in U-Boot on Target

These steps will show how to update the eMMC via a network.

Tip

A working network is necessary! *Setup Network Host*

- Load your image via network to RAM:

```
u-boot=> dhcp ${loadaddr} phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic
BOOTP broadcast 1
DHCP client bound to address 192.168.3.11 (101 ms)
Using ethernet@30be0000 device
TFTP from server 192.168.3.10; our IP address is 192.168.3.11
Filename 'phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic'.
Load address: 0x40480000
Loading: #####################################
         #####################################
         #####################################
         ...
         ...
         ...
         #####################################
         #############
         11.2 MiB/s
done
Bytes transferred = 911842304 (36599c00 hex)
```

- Write the image to the eMMC:

```
u-boot=> mmc dev 2
switch to partitions #0, OK
mmc2(part 0) is current device
u-boot=> setexpr nblk ${filesize} / 0x200
u-boot=> mmc write ${loadaddr} 0x0 ${nblk}

MMC write: dev # 2, block # 0, count 1780942 ... 1780942 blocks written: OK
```

4.2.1.2 Flash eMMC via Network in Linux on Target

You can update the eMMC from your target.

Tip

A working network is necessary! *Setup Network Host*

Take a compressed or uncompressed image on the host and send it with ssh through the network (then uncompress it, if necessary) to the eMMC of the target with a one-line command:

```
target:~$ ssh <USER>@192.168.3.10 "dd if=<path_to_file>/phytec-qt6demo-image-phyboard-pollux-
↪imx8mp-3.rootfs.wic" | dd of=/dev/mmcblk2 conv=fsync
```

4.2.1.3 Flash eMMC via Network in Linux on Host

It is also possible to install the OS at eMMC from your Linux host. As before, you need a complete image on your host.

Tip

A working network is necessary! *Setup Network Host*

Show your available image files on the host:

```
host:~$ ls
phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic
```

Send the image with the `dd` command combined with ssh through the network to the eMMC of your device:

```
host:~$ dd if=phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic status=progress | ssh␣
↪root@192.168.3.11 "dd of=/dev/mmcblk2 conv=fsync"
```

4.2.2 4.2.2 Flash eMMC U-Boot image via Network from running U-Boot

Update the standalone U-Boot image imx-boot is also possible from U-Boot. This can be used if the bootloader on eMMC is located in the eMMC user area.

Tip

A working network is necessary! *Setup Network Host*

Load image over tftp into RAM and then write it to eMMC:

```
u-boot=> dhcp ${loadaddr} imx-boot
u-boot=> setexpr nblk ${filesize} / 0x200
u-boot=> mmc dev 2
u-boot=> mmc write ${loadaddr} 0x40 ${nblk}
```

Hint

The hexadecimal value represents the offset as a multiple of 512 byte blocks. See the *offset table* for the correct value of the corresponding SoC.

4.2.3 4.2.3 Flash eMMC from USB

4.2.3.1 Flash eMMC from USB in U-Boot on Target

Note

Only the lower USB-A port is configured for storage devices and only this port will work when trying to access a storage device in U-Boot.

Tip

This step only works if the size of the image file is less than 1GB due to limited usage of RAM size in the Bootloader after enabling OPTEE.

These steps will show how to update the eMMC via a USB device. Configure the *bootmode switch (S3)* to SD Card and insert an SD card. Power on the board and stop in U-Boot prompt. Insert a USB device with the copied WIC image to the USB slot.

Load your image from the USB device to RAM:

```
u-boot=> usb start
starting USB...
USB0:   USB EHCI 1.00
scanning bus 0 for devices... 2 USB Device(s) found
       scanning usb for storage devices... 1 Storage Device(s) found
u-boot=> fatload usb 0:1 ${loadaddr} *.wic
497444864 bytes read in 31577 ms (15 MiB/s)
```

Write the image to the eMMC:

```
u-boot=> mmc dev 2
switch to partitions #0, OK
mmc2(part 0) is current device
u-boot=> setexpr nblk ${filesize} / 0x200
u-boot=> mmc write ${loadaddr} 0x0 ${nblk}

MMC write: dev # 2, block # 0, count 1024000 ... 1024000 blocks written: OK
u-boot=> boot
```

4.2.3.2 Flash eMMC from USB in Linux

These steps will show how to flash the eMMC on Linux with a USB stick. You only need a complete image saved on the USB stick and a bootable WIC image. (e.g. phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic). Set the *bootmode switch (S3)* to SD Card.

- Insert and mount the USB stick:

```
[   60.458908] usb-storage 1-1.1:1.0: USB Mass Storage device detected
[   60.467286] scsi host0: usb-storage 1-1.1:1.0
[   61.504607] scsi 0:0:0:0: Direct-Access                              8.07 PQ: 0 ANSI: 2
[   61.515283] sd 0:0:0:0: [sda] 3782656 512-byte logical blocks: (1.94 GB/1.80 GiB)
[   61.523285] sd 0:0:0:0: [sda] Write Protect is off
[   61.528509] sd 0:0:0:0: [sda] No Caching mode page found
[   61.533889] sd 0:0:0:0: [sda] Assuming drive cache: write through
[   61.665969]  sda: sda1
[   61.672284] sd 0:0:0:0: [sda] Attached SCSI removable disk
target:~$ mount /dev/sda1 /mnt
```

- Now show your saved image files on the USB Stick:

```
target:~$ cd /mnt
target:~$ ls
phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic
```

- Show list of available MMC devices:

```
target:~$ ls /dev | grep mmc
mmcblk1
mmcblk1p1
mmcblk1p2
mmcblk2
mmcblk2boot0
mmcblk2boot1
mmcblk2p1
mmcblk2p2
mmcblk2rpmb
```

- The eMMC device can be recognized by the fact that it contains two boot partitions: (mmcblk2boot0; mmcblk2boot1)
- Write the image to the phyCORE-i.MX 8M Plus eMMC (MMC device 2 without partition):

```
target:~$ dd if=phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic of=/dev/mmcblk2␣
↪conv=fsync
```

- After a complete write, your board can boot from eMMC.

Tip

Before this will work, you need to configure the *bootmode switch (S3)* to **eMMC**.

4.2.4 4.2.4 Flash eMMC from SD Card

Even if there is no network available, you can update the eMMC. For that, you only need a ready-to-use image file (`*.wic`) located on the SD card. Because the image file is quite large, you have to create a third partition. To create a new partition or enlarge your SD card, see Resizing ext4 Root Filesystem.

Alternatively, flash a partup package to the SD card, as described in *Getting Started*. This will ensure the full space of the SD card is used.

4.2.4.1 Flash eMMC from SD card in U-Boot on Target

Tip

This step only works if the size of the image file is less than 1GB due to limited usage of RAM size in the Bootloader after enabling OPTEE. If the image file is too large use the *Updating eMMC from SD card in Linux on Target* subsection.

- Flash an SD card with a working image and create a third FAT partition. Copy the WIC image (for example phytec-qt6demo-image.rootfs.wic) to this partition.
- Configure the *bootmode switch (S3)* to SD Card and insert the SD Card.
- Power on the board and stop in U-Boot.
- Load the image:

```
u-boot=> fatload mmc 1:3 ${loadaddr} phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.
↪wic
reading
911842304 bytes read in 39253 ms (22.2 MiB/s)
```

- Switch the mmc dev to eMMC:

```
u-boot=> mmc list
FSL_SDHC: 1 (SD)
FSL_SDHC: 2 (eMMC)
u-boot=> mmc dev 2
switch to partitions #0, OK
mmc2(part 0) is current device
```

- Flash your WIC image (for example phytec-qt6demo-image.rootfs.wic) from the SD card to eMMC. This will partition the card and copy imx-boot, Image, dtb, dtbo, and root file system to eMMC.

```
u-boot=> setexpr nblk ${filesize} / 0x200
u-boot=> mmc write ${loadaddr} 0x0 ${nblk}

MMC write: dev # 2, block # 0, count 1780942 ... 1780942 blocks written: OK
```

- Power off the board and change the *bootmode switch (S3)* to eMMC.

4.2.4.2 Flash eMMC from SD card in Linux on Target

You can also flash the eMMC on Linux. You only need a partup package or WIC image saved on the SD card.

- Show your saved partup package or WIC image files on the SD card:

```
target:~$ ls
phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.partup
phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic
```

- Show list of available MMC devices:

```
target:~$ ls /dev | grep mmc
mmcblk1
mmcblk1p1
mmcblk1p2
mmcblk2
mmcblk2boot0
mmcblk2boot1
mmcblk2p1
mmcblk2p2
mmcblk2rpmb
```

- The eMMC device can be recognized by the fact that it contains two boot partitions: (mmcblk2**boot0**; mmcblk2**boot1**)
- Write the image to the phyCORE-i.MX 8M Plus eMMC (MMC device 2 **without** partition) using partup:

```
target:~$ partup install phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.partup /dev/
↪mmcblk2
```

Flashing the partup package has the advantage of using the full capacity of the eMMC device, adjusting partitions accordingly.

Note

Alternatively, `dd` may be used instead:

```
target:~$ dd if=phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic of=/dev/mmcblk2␣
↪conv=fsync
```

Keep in mind that the root partition does not make use of the full space when flashing with `dd`.

- After a complete write, your board can boot from eMMC.

Warning

Before this will work, you need to configure the *bootmode switch (S3)* to eMMC.

4.3 4.3 RAUC

The RAUC (Robust Auto-Update Controller) mechanism support has been added to meta-ampliphy. It controls the procedure of updating a device with new firmware. This includes updating the Linux kernel, Device Tree, and root filesystem. PHYTEC has written an online manual on how we have integrated RAUC into our BSPs: L-1006e.A6 RAUC Update & Device Management Manual.

CHAPTER

FIVE

5 DEVELOPMENT

5.1 5.1 Host Network Preparation

For various tasks involving a network in the Bootloader, some host services are required to be set up. On the development host, a TFTP, NFS and DHCP server must be installed and configured. The following tools will be needed to boot via Ethernet:

```
host:~$ sudo apt install tftpd-hpa nfs-kernel-server kea
```

5.1.1 5.1.1 TFTP Server Setup

- First, create a directory to store the TFTP files:

  ```
  host:~$ sudo mkdir /srv/tftp
  ```

- Then copy your BSP image files to this directory and make sure other users have read access to all the files in the tftp directory, otherwise they are not accessible from the target.

  ```
  host:~$ sudo chmod -R o+r /srv/tftp
  ```

- You also need to configure a static IP address for the appropriate interface. The default IP address of the PHYTEC evaluation boards is 192.168.3.11. Setting a host address 192.168.3.10 with netmask 255.255.255.0 is a good choice.

  ```
  host:~$ ip addr show <network-interface>
  ```

  Replace <network-interface> with the network interface you configured and want to connect the board to. You can show all network interfaces by not specifying a network interface.

- The message you receive should contain this:

  ```
  inet 192.168.3.10/24 brd 192.168.3.255
  ```

- Create or edit the `/etc/default/tftpd-hpa` file:

  ```
  # /etc/default/tftpd-hpa

  TFTP_USERNAME="tftp"
  TFTP_DIRECTORY="/srv/tftp"
  TFTP_ADDRESS=":69"
  TFTP_OPTIONS="-s -c"
  ```

- Set TFTP_DIRECTORY to your TFTP server root directory

- Set TFTP_ADDRESS to the host address the server is listening to (set to 0.0.0.0:69 to listen to all local IPs)
- Set TFTP_OPTIONS, the following command shows the available options:

```
host:~$ man tftpd
```

- Restart the services to pick up the configuration changes:

```
host:~$ sudo service tftpd-hpa restart
```

Now connect the ethernet port of the board to your host system. We also need a network connection between the embedded board and the TFTP server. The server should be set to IP 192.168.3.10 and netmask 255.255.255.0.

5.1.1.1 NFS Server Setup

- Create an nfs directory:

```
host:~$ sudo mkdir /srv/nfs
```

- Temporarily export the nfs directory: The NFS server is not restricted to a certain file system location, so all we have to do is to export our root file system to the embedded network. In this example, the whole directory is exported and the "lab network" address of the development host is 192.168.3.10. The IP address has to be adapted to the local needs:

```
host:~$ sudo exportfs -i -o rw,no_root_squash,sync,no_subtree_check 192.168.3.0/
↪255.255.255.0:/srv/nfs
```

- unexport the rootfs when finished:

```
host:~$ sudo exportfs -u 192.168.3.0/255.255.255.0:/srv/nfs
```

5.1.1.1.1 Permanent export

- To make the export persistent across reboots on most distributions, modify the `/etc/exports` file and export it:

```
/srv/nfs 192.168.3.0/255.255.255.0(rw,no_root_squash,sync,no_subtree_check)
```

- Now the NFS-Server has to read the `/etc/exportfs` file again:

```
host:~$ sudo exportfs -ra
```

5.1.1.2 DHCP Server setup

- Create or edit the `/etc/kea/kea-dhcp4.conf` file; Using the internal subnet sample. Replace <network-interface> with the name for the physical network interface:

```
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [ "<network-interface>/192.168.3.10" ]
    },
    "lease-database": {
```

(continues on next page)

(continued from previous page)

```
      "type": "memfile",
      "persist": true,
      "name": "/tmp/dhcp4.leases"
    },
    "valid-lifetime": 28800,
    "subnet4": [{
        "id": 1,
        "next-server": "192.168.3.10",
        "subnet": "192.168.3.0/24",
        "pools": [
          { "pool": "192.168.3.1 - 192.168.3.255" }
        ]
    }]
  }
}
```

Warning

Be careful when creating subnets as this may interfere with the company network policy. To be on the safe side, use a different network and specify that via the `interfaces` configuration option.

- Now the DHCP-Server has to read the `/etc/kea/kea-dhcp4.conf` file again:

```
host:~$ sudo systemctl restart kea-dhcp4-server
```

When you boot/restart your host PC and don't have the network interface, as specified in the kea-dhcp4 config, already active the kea-dhcp4-server will fail to start. Make sure to start/restart the systemd service when you connect the interface.

Note

DHCP server setup is only needed when using dynamic IP addresses. For our vendor BSPs, static IP addresses are used by default.

```
u-boot=> env print ip_dyn
ip_dyn=no
```

To use dynamic IP addresses for netboot, `ip_dyn` needs to be set to `yes`.

5.2 5.2 Booting the Kernel from a Network

Booting from a network means loading the kernel and device tree over TFTP and the root file system over NFS. The bootloader itself must already be loaded from another available boot device.

5.2.1 5.2.1 Place Images on Host for Netboot

- Copy the kernel image to your tftp directory:

```
host:~$ cp Image /srv/tftp
```

- Copy the devicetree to your tftp directory:

```
host:~$ cp oftree /srv/tftp
```

- Make sure other users have read access to all the files in the tftp directory, otherwise they are not accessible from the target:

```
host:~$ sudo chmod -R o+r /srv/tftp
```

- Extract the rootfs to your nfs directory:

```
host:~$ sudo tar -xvzf phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.tar.gz -C /srv/
↪nfs
```

Note

Make sure you extract with sudo to preserve the correct ownership.

5.2.2 5.2.2 Booting from an Embedded Board

Boot the board into the U-boot prompt and press any key to hold.

- To boot from a network, call:

```
u-boot=> run netboot
```

5.3 5.3 Working with UUU-Tool

The Universal Update Utility Tool (UUU-Tool) from NXP is a software to execute on the host to load and run the bootloader on the board through SDP (Serial Download Protocol). For detailed information visit https://github.com/nxp-imx/mfgtools or download the Official UUU-tool documentation.

5.3.1 5.3.1 Host preparations for UUU-Tool Usage

- Follow the instructions from https://github.com/nxp-imx/mfgtools#linux.
- If you built UUU from source, add it to `PATH`:

  This BASH command adds UUU only temporarily to `PATH`. To add it permanently, add this line to `~/.bashrc`.

```
export PATH=~/mfgtools/uuu/:"$PATH"
```

- Set udev rules (documented in `uuu -udev`):

```
host:~$ sudo sh -c "uuu -udev >> /etc/udev/rules.d/70-uuu.rules"
host:~$ sudo udevadm control --reload
```

5.3.2 5.3.2 Get Images

Download imx-boot from our server or get it from your Yocto build directory at build/deploy/images/phyboard-pollux-imx8mp-3/. For flashing a wic image to eMMC, you will also need phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic

5.3.3 5.3.3 Prepare Target

Set the *bootmode switch (S3)* to **USB Serial Download**. Also, connect USB port *X5 (upper connector)* to your host.

5.3.4 5.3.4 Starting bootloader via UUU-Tool

Execute and power up the board:

```
host:~$ sudo uuu -b spl imx-boot
```

You can see the bootlog on the console via *(X1)*, as usual.

Note

The default boot command when booting with UUU-Tool is set to fastboot. If you want to change this, please adjust the environment variable bootcmd_mfg in U-boot prompt with setenv bootcmd_mfg. Please note, when booting with UUU-tool the default environment is loaded. Saveenv has no effect. If you want to change the boot command permanently for UUU-boot, you need to change this in U-Boot code.

5.3.5 5.3.5 Flashing U-boot Image to eMMC via UUU-Tool

Warning

UUU flashes U-boot into eMMC BOOT (hardware) boot partitions, and it sets the BOOT_PARTITION_ENABLE in the eMMC! This is a problem since we want the bootloader to reside in the eMMC USER partition. Flashing next U-Boot version .wic image and not disabling BOOT_PARTITION_ENABLE bit will result in device always using U-boot saved in BOOT partitions. To fix this in U-Boot:

```
u-boot=> mmc partconf 2 0 0 0
u-boot=> mmc partconf 2
EXT_CSD[179], PARTITION_CONFIG:
BOOT_ACK: 0x0
BOOT_PARTITION_ENABLE: 0x0
PARTITION_ACCESS: 0x0
```

or check Disable booting from eMMC boot partitions from Linux.

This way the bootloader is still flashed to eMMC BOOT partitions but it is not used!

When using **partup** tool and `.partup` package for eMMC flashing this is done by default, which makes partup again superior flash option.

Execute and power up the board:

```
host:~$ sudo uuu -b emmc imx-boot
```

5.3.6 5.3.6 Flashing wic Image to eMMC via UUU-Tool

Execute and power up the board:

```
host:~$ sudo uuu -b emmc_all imx-boot phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic
```

5.4 5.4 Standalone Build preparation

In this section, we describe how to build the U-Boot and the Linux kernel without using the Yocto Project. This procedure makes the most sense for development. The U-Boot source code, the Linux kernel, and all other git repositories are available on GitHub.

5.4.1 5.4.1 Git Repositories

- Used U-Boot repository:

```
https://github.com/phytec/u-boot-phytec.git
```

- Our U-Boot is based on the u-boot-phytec and adds board-specific patches.
- Used Linux kernel repository:

```
https://github.com/phytec/linux-phytec.git
```

- Our i.MX 8M Plus kernel is based on the linux-phytec kernel.

To find out which u-boot and kernel tags to use for a specific board, have a look at your BSP source folder:

```
recipes-kernel/linux/linux-phytec_*.bb
meta-phytec/recipes-bsp/u-boot/u-boot-phytec_*.bb
```

5.4.2 5.4.2 Build the SDK

You can build the SDK yourself with Yocto:

- Move to the Yocto build directory:

```
host:~$ source sources/poky/oe-init-build-env
host:~$ bitbake -c populate_sdk phytec-qt6demo-image # or another image
```

5.4.3 5.4.3 Install the SDK

- Set correct permissions and install the SDK:

```
host:~$ chmod +x phytec-ampliphy-xwayland-glibc-x86_64-phytec-qt6demo-image-cortexa53-
↪crypto-toolchain-4.3.sh
host:~$ ./phytec-ampliphy-xwayland-glibc-x86_64-phytec-qt6demo-image-cortexa53-crypto-
↪toolchain-4.3.sh
==========================================================================================================
Enter target directory for SDK (default: /opt/ampliphy-xwayland/4.3):
You are about to install the SDK to "/opt/ampliphy-xwayland/4.3". Proceed [Y/n]? Y
Extracting SDK...done
Setting it up...done
SDK has been successfully set up and is ready to be used.
```

5.4.4 5.4.4 Using the SDK

Activate the toolchain for your shell by sourcing the *environment-setup* file in the toolchain directory:

```
host:~$ source /opt/ampliphy-xwayland/4.3/environment-setup-cortexa53-crypto-phytec-linux
```

5.4.5 5.4.5 Installing Required Tools

Building Linux and U-Boot out-of-tree requires some additional host tool dependencies to be installed. For Ubuntu you can install them with:

```
host:~$ sudo apt install bison flex libssl-dev
```

5.5 5.5 U-Boot standalone build

5.5.1 5.5.1 Get the source code

- Get the U-Boot sources:

  ```
  host:~$ git clone https://github.com/phytec/u-boot-phytec.git
  ```

- To get the correct *U-Boot* **tag** you need to take a look at our release notes, which can be found here: release notes
- The **tag** used in this release is called v2024.01-phy3
- Check out the needed *U-Boot* **tag**:

  ```
  host:~$ cd ~/u-boot-phytec/
  host:~/u-boot-phytec$ git fetch --all --tags
  host:~/u-boot-phytec$ git checkout tags/v2024.01-phy3
  ```

- Set up a build environment:

  ```
  host:~/u-boot-phytec$ source /opt/ampliphy-xwayland/4.3/environment-setup-cortexa53-crypto-
  ↪phytec-linux
  ```

5.5.2 5.5.2 Get the needed binaries

To build the bootloader, you need to **copy** these **files** to your u-boot-phytec **build directory** and rename them to fit with *mkimage* script:

- **ARM Trusted firmware binary** (*mkimage tool* compatible format **bl31.bin**): bl31-imx8mp.bin
- **OPTEE image** (optional): tee.bin
- **DDR firmware files** (*mkimage tool* compatible format **lpddr4_[i,d]mem_*d_*.bin**): lpddr4_dmem_1d_*.bin, lpddr4_dmem_2d_*.bin, lpddr4_imem_1d_*.bin, lpddr4_imem_2d_*.bin

If you already built our BSP with Yocto, you can get the bl31-imx8mp.bin, tee.bin and lpddr4_*.bin from the directory mentioned here: *BSP Images*

Or you can download the files here: https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX8MP/BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1/images/ampliphy-xwayland/phyboard-pollux-imx8mp-3/

Warning

Make sure you rename the files you need so that they are compatible with the *mkimage tool*.

5.5.3 5.5.3 Build the bootloader

- build flash.bin (imx-boot):

```
host:~/u-boot-phytec$ make phycore-imx8mp_defconfig
host:~/u-boot-phytec$ make flash.bin
```

5.5.4 5.5.4 Flash the bootloader to a block device

The flash.bin can be found at u-boot-phytec/ directory and now can be flashed. A chip-specific offset is needed:

SoC	Offset User Area	Offset Boot Partition	e.MMC Device
i.MX 8M Plus	32 kiB	0 kiB	/dev/mmcblk2

E.g. flash SD card:

```
host:~/u-boot-phytec$ sudo dd if=flash.bin of=/dev/sd[x] bs=1024 seek=32 conv=fsync
```

Hint

The specific offset values are also declared in the Yocto variables “BOOTLOADER_SEEK” and “BOOTLOADER_SEEK_EMMC”

5.6 5.6 Kernel standalone build

Note

DTB Filenames: Command lines listed in this chapter use the device tree file name imx8mp-phyboard-pollux-rdk.dtb as an example. The actual file name always corresponds to the SOM/carrier board combination you want to use. For supported SOM/carrier board combinations, see *Supported Hardware*.

5.6.1 5.6.1 Setup sources

- The used linux-phytec branch can be found in the release notes
- The tag needed for this release is called v6.6.21-phy1
- Check out the needed linux-phytec tag:

```
host:~$ git clone https://github.com/phytec/linux-phytec.git
host:~$ cd ~/linux-phytec/
host:~/linux-phytec$ git fetch --all --tags
host:~/linux-phytec$ git checkout tags/v6.6.21-phy1
```

- For committing changes, it is highly recommended to switch to a new branch:

```
host:~/linux-phytec$ git switch --create <new-branch>
```

- Set up a build environment:

```
host:~/linux-phytec$ source /opt/ampliphy-xwayland/4.3/environment-setup-cortexa53-crypto-
↪phytec-linux
```

5.6.2 5.6.2 Build the kernel

- Build the linux kernel:

```
host:~/linux-phytec$ make defconfig
host:~/linux-phytec$ make -j$(nproc)
```

- Install kernel modules to e.g. NFS directory:

```
host:~/linux-phytec$ make INSTALL_MOD_PATH=/home/<user>/<rootfspath> modules_install
```

- The Image can be found at ~/linux-phytec/arch/arm64/boot/Image
- The dtb can be found at ~/linux-phytec/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dtb
- For (re-)building only Devicetrees and -overlays, it is sufficient to run

```
host:~/linux-phytec$ make dtbs
```

Note

If you are facing the following build issue:

```
scripts/dtc/yamltree.c:9:10: fatal error: yaml.h: No such file or directory
```

Make sure you installed the package *"libyaml-dev"* on your host system:

```
host:~$ sudo apt install libyaml-dev
```

5.6.3 5.6.3 Copy Kernel to SD Card

When one-time boot via netboot is not sufficient, the kernel along with its modules and the corresponding device tree blob may be copied directly to a mounted SD card.

```
host:~/linux-phytec$ cp arch/arm64/boot/Image /path/to/sdcard/boot/
host:~/linux-phytec$ cp arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dtb /path/to/
↪sdcard/boot/oftree
host:~/linux-phytec$ make INSTALL_MOD_PATH=/path/to/sdcard/root/ modules_install
```

5.7 5.7 Accessing the Development states

5.7.1 5.7.1 Development state of current release

These release manifests exist to give you access to the development states of the *Yocto* BSP. They will not be displayed in the phyLinux selection menu but need to be selected manually. This can be done using the

following command line:

```
host:~$ ./phyLinux init -p imx8mp -r BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1
```

This will initialize a BSP that will track the latest development state of the current release (BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1). From now on *repo sync* in this folder will pull all the latest changes from our Git repositories:

```
host:~$ repo sync
```

5.7.2 5.7.2 Development state of upcoming release

Also development states of upcoming releases can be accessed this way. For this execute the following command and look for a release with a higher PDXX.Y number than the latest one (BSP-Yocto-Ampliphy-i.MX8MP-PD24.1.1) and `.y` at the end:

```
host:~$ ./phyLinux init -p imx8mp
```

5.8 5.8 Accessing the Latest Upstream Support

We have a vanilla manifest that makes use of the Yocto master branches (not an NXP release), Linux, and U-Boot. This can be used to test the latest upstream kernel/U-Boot.

Note

The master manifest reflects the latest state of development. This tends to be broken from time to time. We try to fix the master on a regular basis.

```
host:~$ ./phyLinux init -p imx8mp -r BSP-Yocto-Ampliphy-i.MX8MP-master
```

5.9 5.9 Format SD-Card

Most images are larger than the default root partition. To flash any storage device with SD Card, the rootfs needs to be expanded or a separate partition needs to be created. There are some different ways to format the SD Card. The easiest way to do this is to use the UI program Gparted.

5.9.1 5.9.1 Gparted

- Get GParted:

  ```
  host:~$ sudo apt install gparted
  ```

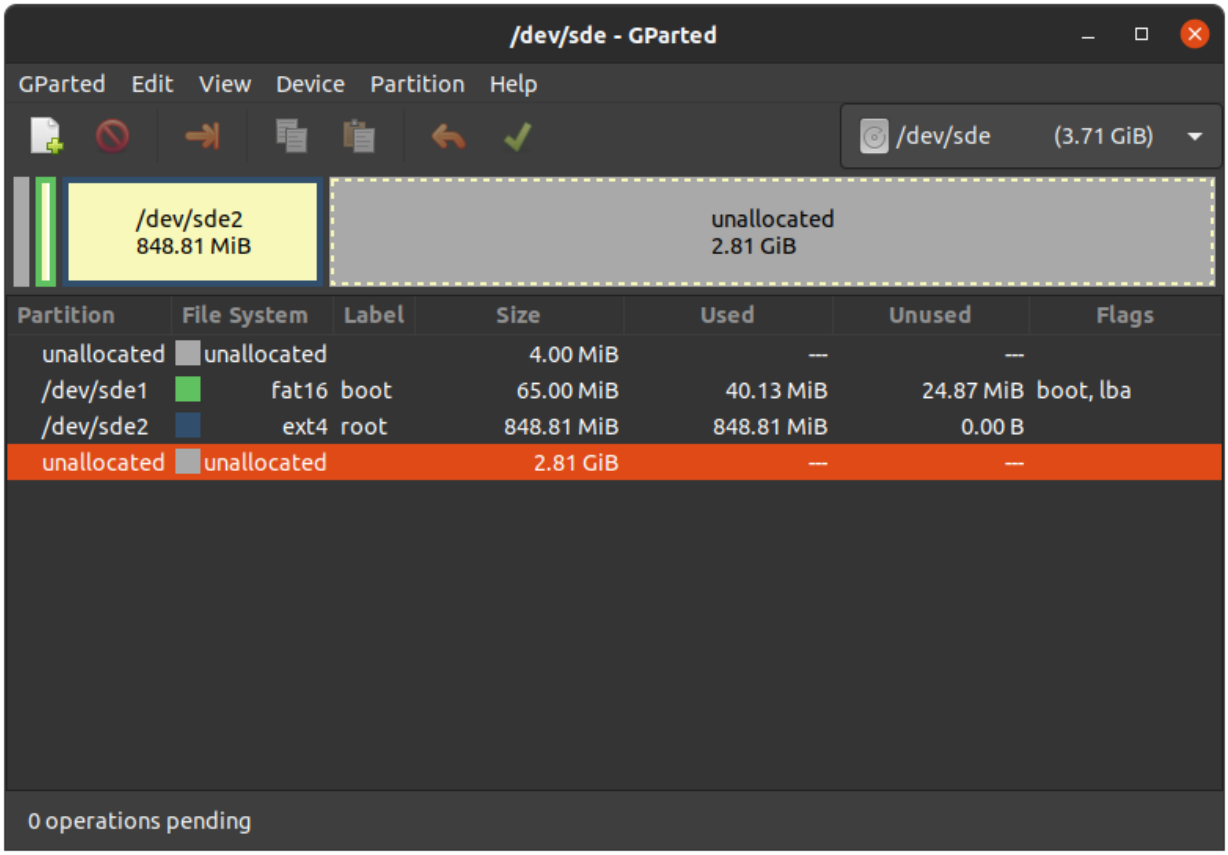
- Insert the SD Card into your host and get the device name:

  ```
  host:~$ dmesg | tail
  ...
  [30436.175412] sd 4:0:0:0: [sdb] 62453760 512-byte logical blocks: (32.0 GB/29.8 GiB)
  [30436.179846]  sdb: sdb1 sdb2
  ...
  ```

- Unmount all SD Card partitions.

- Launch GParted:

```
host:~$ sudo gparted
```

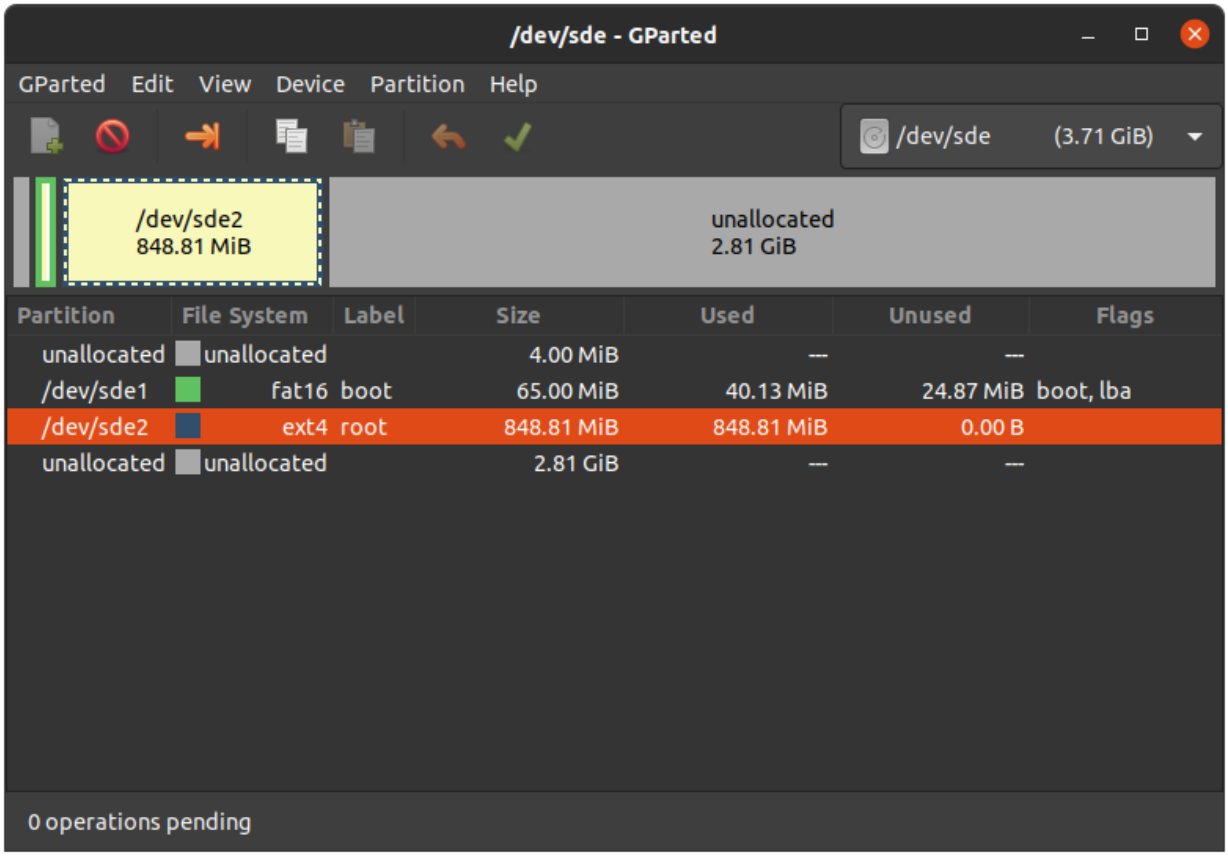


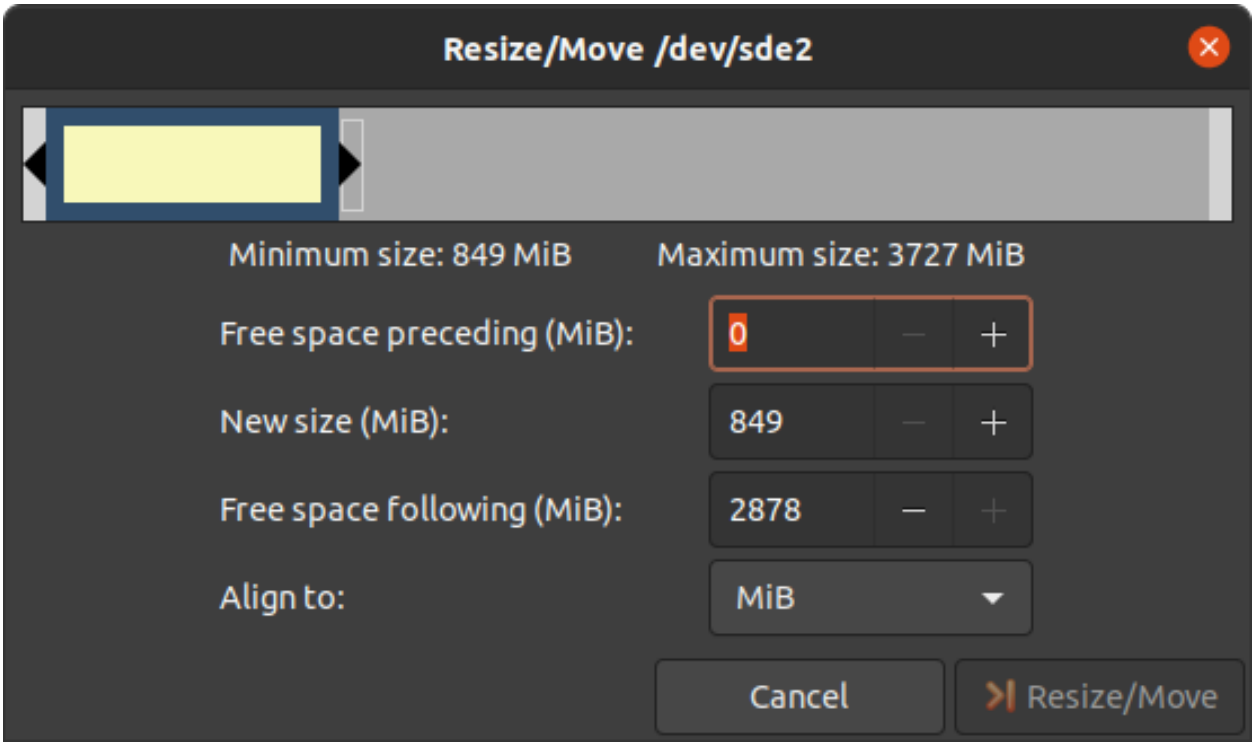
5.9.1.1 Expand rootfs

Warning

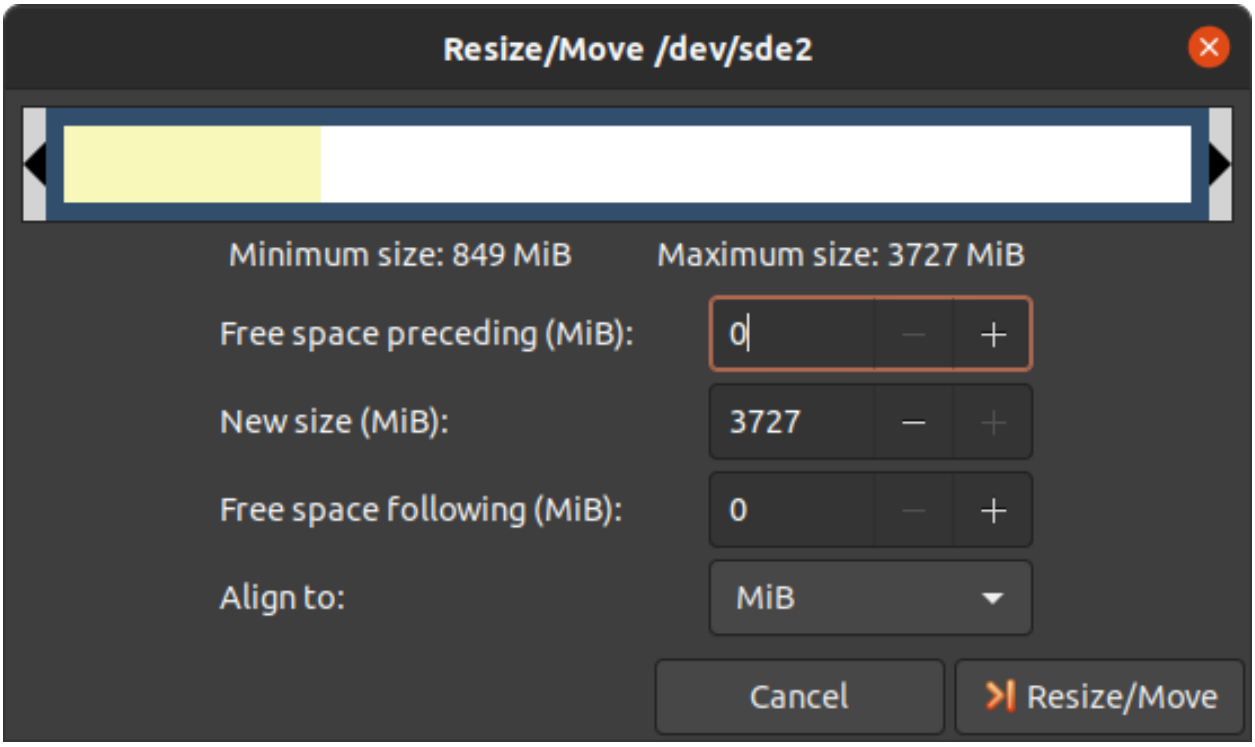
Running gparted on host systems which are using resize2fs version 1.46.6 and older (e.g. Ubuntu 22.04) are not able to expand the ext4 partition created with Yocto Mickledore and newer. This is due to a new default option in resize2fs which causes a incompatibility. See release notes.

- Choose your SD Card device at the drop-down menu on the top right
- Choose the ext4 root partition and click on resize:

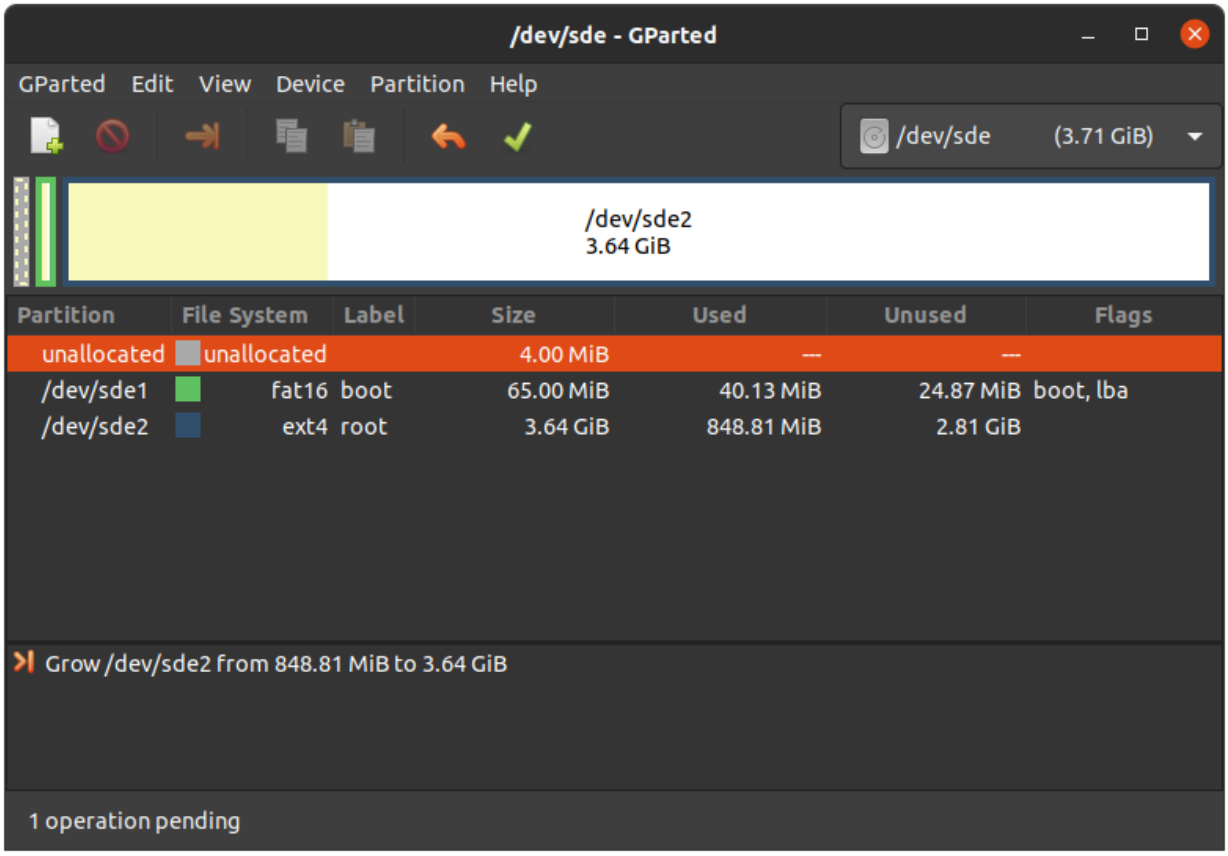




- Drag the slider as far as you like or enter the size manually.



- Confirm your entry by clicking on the “Change size” button.



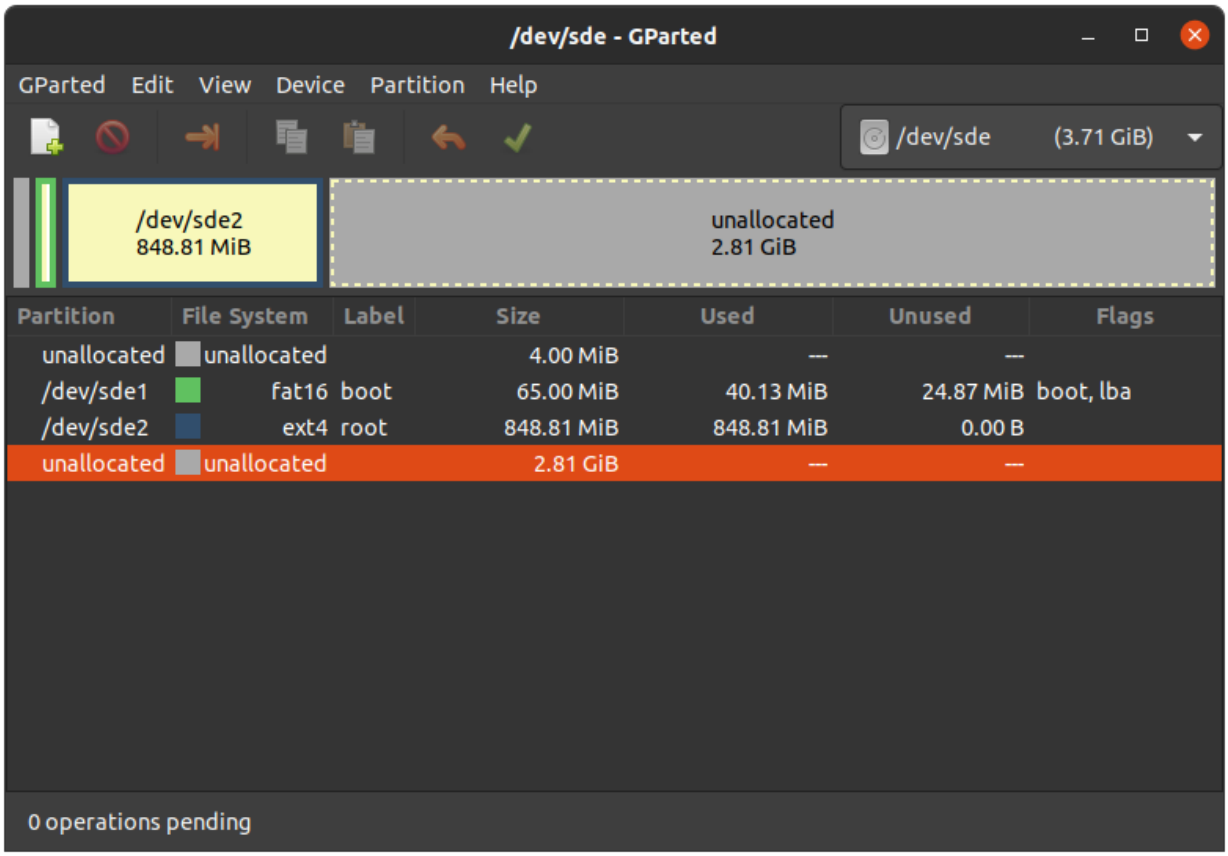
- To apply your changes, press the green tick.

- Now you can mount the root partition and copy e.g. the phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic image to it. Then unmount it again:

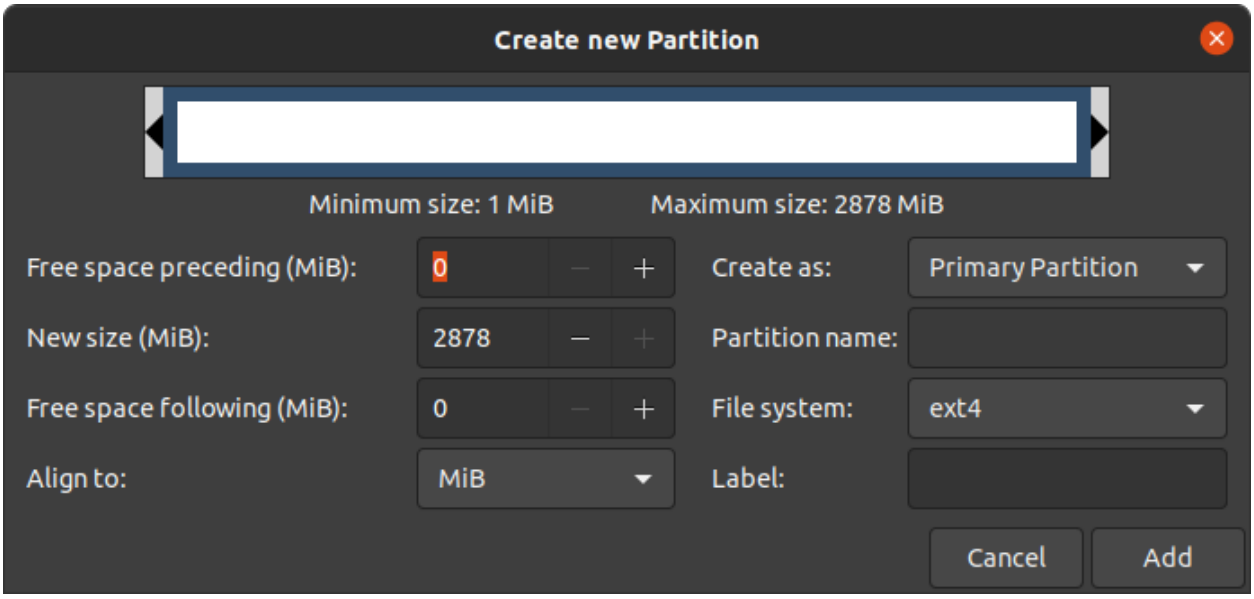
```
host:~$ sudo cp phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic /mnt/ ; sync
host:~$ umount /mnt
```

5.9.1.2 Create the Third Partition

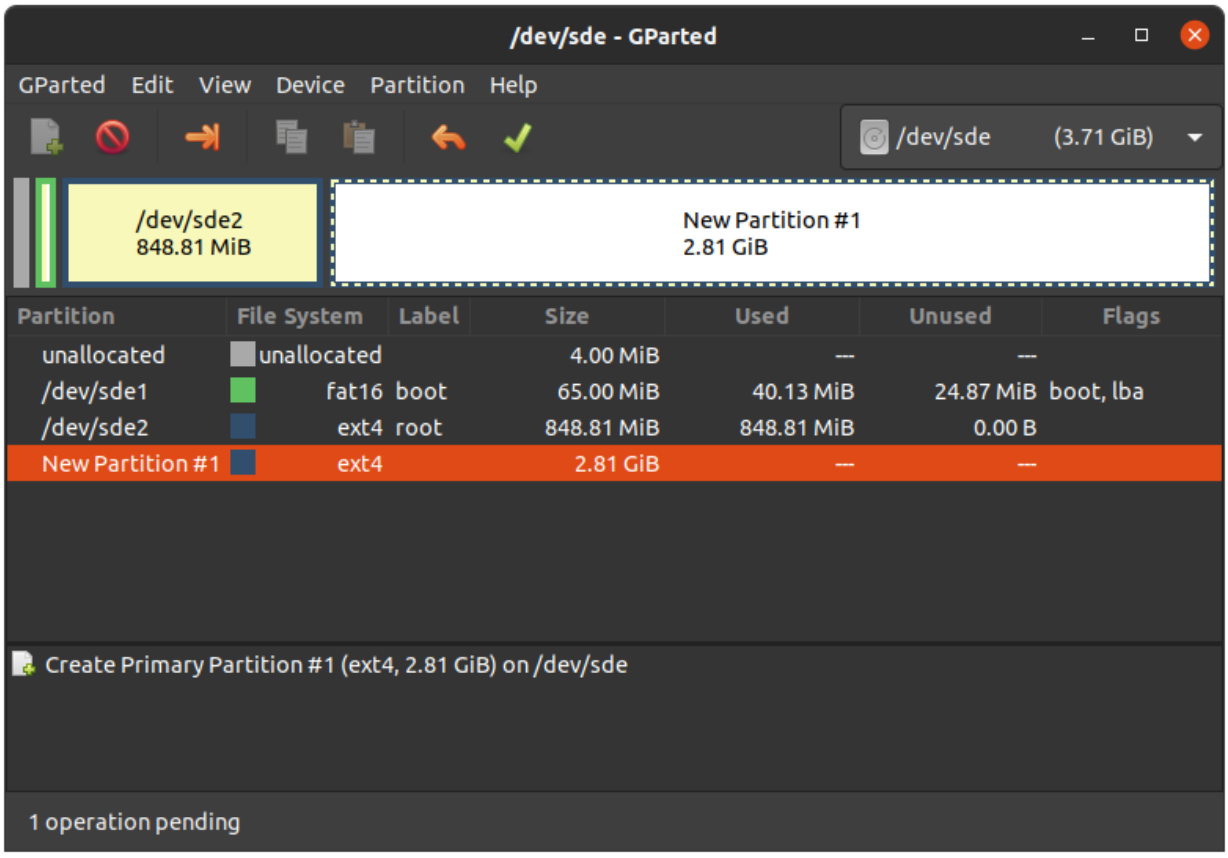
- Choose your SD Card device at the drop-down menu on the top right



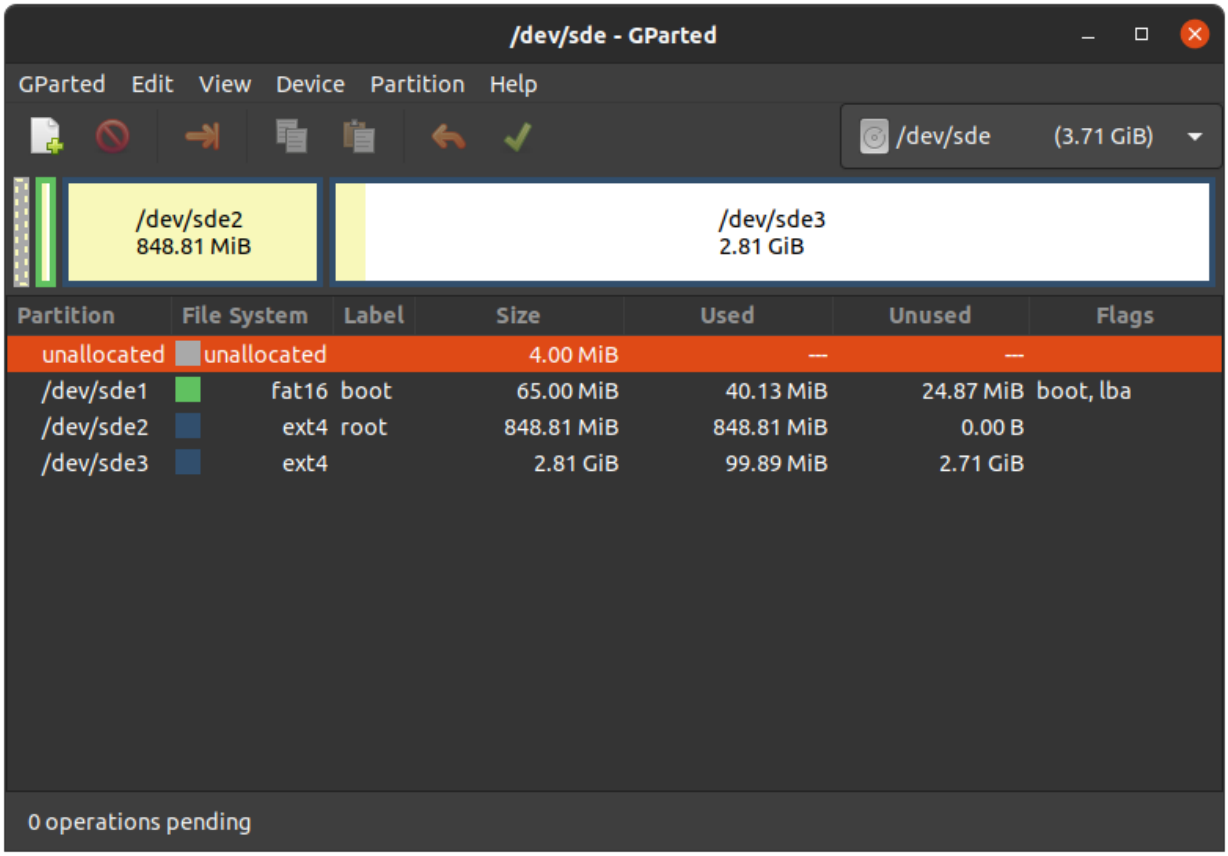
- Choose the bigger unallocated area and press “New”:



- Click “Add”



- Confirm your changes by pressing the green tick.



- Now you can mount the new partition and copy e.g. phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic image to it. Then unmount it again:

```
host:~$ sudo mount /dev/sde3 /mnt
host:~$ sudo cp phytec-qt6demo-image-phyboard-pollux-imx8mp-3.rootfs.wic /mnt/ ; sync
host:~$ umount /mnt
```

CHAPTER

SIX

6 DEVICE TREE (DT)

6.1 6.1 Introduction

The following text briefly describes the Device Tree and can be found in the Linux kernel Documentation (https://docs.kernel.org/devicetree/usage-model.html)

"The "Open Firmware Device Tree", or simply Devicetree (DT), is a data structure and language for describing hardware. More specifically, it is a description of hardware that is readable by an operating system so that the operating system doesn't need to hard code details of the machine."

The kernel documentation is a really good source for a DT introduction. An overview of the device tree data format can be found on the device tree usage page at devicetree.org.

6.2 6.2 PHYTEC i.MX 8M Plus BSP Device Tree Concept

The following sections explain some rules PHYTEC has defined on how to set up device trees for our i.MX 8M Plus SoC-based boards.

6.2.1 6.2.1 Device Tree Structure

- *Module.dtsi* - Module includes all devices mounted on the SoM, such as PMIC and RAM.
- *Board.dts* - include the module dtsi file. Devices that come from the i.MX 8M Plus SoC but are just routed down to the carrier board and used there are included in this dts.
- *Overlay.dtso* - enable/disable features depending on optional hardware that may be mounted or missing on SoM or baseboard (e.g SPI flash or PEB-AV-10)

From the root directory of the Linux Kernel our devicetree files for i.MX 8 platforms can be found in `arch/arm64/boot/dts/freescale/`.

6.2.2 6.2.2 Device Tree Overlay

Device Tree overlays are device tree fragments that can be merged into a device tree during boot time. These are for example hardware descriptions of an expansion board. They are instead of being added to the device tree as an extra include, now applied as an overlay. They also may only contain setting the status of a node depending on if it is mounted or not. The device tree overlays are placed next to the other device tree files in our Linux kernel repository in the folder `arch/arm64/boot/dts/freescale/`.

Available overlays for phyboard-pollux-imx8mp-3.conf are:

CHAPTER

SEVEN

7 ACCESSING PERIPHERALS

To find out which boards and modules are supported by the release of PHYTEC's phyCORE-i.MX 8M Plus BSP described herein, visit our BSP web page and click the corresponding BSP release in the download section. Here you can find all hardware supported in the columns "Hardware Article Number" and the correct machine name in the corresponding cell under "Machine Name".

To achieve maximum software reuse, the Linux kernel offers a sophisticated infrastructure that layers software components into board-specific parts. The BSP tries to modularize the kit features as much as possible. When a customized baseboard or even a customer-specific module is developed, most of the software support can be reused without error-prone copy-and-paste. The kernel code corresponding to the boards can be found in device trees (DT) in the kernel repository under `arch/arm64/boot/dts/freescale/*.dts`.

In fact, software reuse is one of the most important features of the Linux kernel, especially of the ARM implementation which always has to fight with an insane number of possibilities of the System-on-Chip CPUs. The whole board-specific hardware is described in DTs and is not part of the kernel image itself. The hardware description is in its own separate binary, called the Device Tree Blob (DTB) (section *device tree*).

Please read section PHYTEC i.MX 8M Plus BSP Device Tree Concept to get an understanding of our i.MX 8 BSP device tree model.

The following sections provide an overview of the supported hardware components and their operating system drivers on the i.MX 8 platform. Further changes can be ported upon customer request.

7.1 7.1 i.MX 8M Plus Pin Muxing

The i.MX 8M Plus SoC contains many peripheral interfaces. In order to reduce package size and lower overall system cost while maintaining maximum functionality, many of the i.MX 8M Plus terminals can multiplex up to eight signal functions. Although there are many combinations of pin multiplexing that are possible, only a certain number of sets, called IO sets, are valid due to timing limitations. These valid IO sets were carefully chosen to provide many possible application scenarios for the user.

Please refer to our Hardware Manual or the NXP i.MX 8M Plus Reference Manual for more information about the specific pins and the muxing capabilities.

The IO set configuration, also called muxing, is done in the Device Tree. The driver pinctrl-single reads the DT's node fsl,pins, and does the appropriate pin muxing.

The following is an example of the pin muxing of the UART1 device in imx8mp-phyboard-pollux-rdk.dts:

```
pinctrl_uart1: uart1grp {
        fsl,pins = <
                MX8MP_IOMUXC_UART1_RXD_UART1_DCE_RX     0x140
                MX8MP_IOMUXC_UART1_TXD_UART1_DCE_TX     0x140
        >;
};
```

The first part of the string MX8MP_IOMUXC_UART1_RXD_UART1_DCE_RX names the pad (in this example UART1_RXD). The second part of the string (UART1_DCE_RX) is the desired muxing option for this pad. The pad setting value (hex value on the right) defines different modes of the pad, for example, if internal pull resistors are activated or not. In this case, the internal resistors are disabled.

The device tree representation for UART1 pinmuxing: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L387

7.2 7.2 RS232/RS485

The phyCORE-i.MX 8M Plus supports up to 4 UART units. On the phyBOARD-Pollux, TTL level signals of UART1 (the standard console) and UART4 are routed to Silicon Labs CP2105 UART to USB converter expansion. This USB is brought out at Micro-USB connector X1. UART3 is at X6 (Expansion Connector) at TTL level. UART2 is connected to a multi-protocol transceiver for RS-232 and RS-485, available at pin header connector *X2* at the RS-232 level, or at the RS-485 level. The configuration of the multi-protocol transceiver is done by jumpers *JP3* and *JP4* on the baseboard. For more information about the correct setup please refer to the phyCORE-i.MX 8M Plus/phyBOARD-Pollux Hardware Manual section UARTs.

We use the same device tree node for RS-232 and RS-485. RS-485 mode can be enabled with ioctl TIOCSRS485. Also, full-duplex support is also configured using ioctls. Have a look at our small example application rs485test, which is also included in the BSP. The jumpers *JP3* and *JP4* need to be set correctly.

7.2.1 7.2.1 RS232

- Display the current settings of a terminal in a human-readable format:

```
target:~$ stty -a
```

- Configuration of the UART interface can be done with stty. For example:

```
target:~$ stty -F /dev/ttymxc1 115200 crtscts raw -echo
```

- With a simple echo and cat, basic communication can be tested. Example:

```
target:~$ echo 123 > /dev/ttymxc1
```

```
host:~$ cat /dev/ttyUSB2
```

The host should print out “123”.

7.2.2 7.2.2 RS485

Hint

Remember to use bus termination resistors of 120 Ohm at each end of the bus, when using longer cables.

For easy testing, look at the linux-serial-test. This tool is called the IOCTL for RS485 and sends a constant stream of data.

```
target:~$ linux-serial-test -p /dev/ttymxc1 -b 115200 --rs485 0
```

More information about the linux-serial-test tool and its parameters can be found here: linux-serial-test

The linux-serial-test will automatically set ioctls, but they can also be set manually with rs485conf.

You can show the current config with:

```
target:~$ rs485conf /dev/ttymxc1
```

You can show all options with:

```
target:~$ rs485conf /dev/ttymxc1 -h
```

Documentation for calling the IOCTL within c-code is described in the Linux kernel documentation: https://www.kernel.org/doc/Documentation/serial/serial-rs485.txt

The device tree representation for RS232 and RS485: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L251

7.3 7.3 Ethernet

phyBOARD-Pollux-i.MX 8M Plus provides two ethernet interfaces. A gigabit Ethernet is provided by our module and board.

Warning

The naming convention of the Ethernet interfaces in the hardware (ethernet0 and ethernet1) do not align with the network interfaces (eth0 and eth1) in Linux. So, be aware of these differences:

ethernet1 = eth0
ethernet0 = eth1

All interfaces offer a standard Linux network port that can be programmed using the BSD socket interface. The whole network configuration is handled by the systemd-networkd daemon. The relevant configuration files can be found on the target in `/lib/systemd/network/` as well as the BSP in `meta-ampliphy/recipes-core/systemd/systemd-conf`.

IP addresses can be configured within *.network files. The interfaces are configured to static IP as default. The default IP address and netmask for eth0 is:

```
eth0: 192.168.3.11/24
```

To configure eth0 to dynamic IP over DHCP, go to `/lib/systemd/network/\*-eth0.network` and delete the line:

```
Address=192.168.3.11/24
```

The DT Ethernet setup might be split into two files depending on your hardware configuration: the module DT and the board-specific DT. The device tree set up for the ethernet where the PHY is populated on the SoM can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L41.

The device tree set up for EQOS Ethernet IP core where the PHY is populated on the phyBOARD-Pollux can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L106

7.3.1 7.3.1 Network Environment Customization

7.3.1.1 U-boot network-environment

- We currently use dynamic IP addresses in U-Boot. This is enabled by this variable:

```
u-boot=> printenv ip_dyn
ip_dyn=yes
```

- Set up path for NFS. A modification could look like this:

```
u-boot=> setenv nfsroot /home/user/nfssrc
```

Please note that these modifications will only affect the bootloader settings.

7.3.1.2 Kernel network-environment

- Find the ethernet settings for eth0 in the target kernel:

```
target:~$ ip -statistics address show eth0
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state UP group default qlen␣
↪1000
    link/ether 50:2d:f4:19:d6:33 brd ff:ff:ff:ff:ff:ff
    RX:  bytes packets errors dropped  missed   mcast
               0       0      0       0       0       0
    TX:  bytes packets errors dropped carrier collsns
               0       0      0       0       0       0
```

- Temporary adaption of the eth0 configuration:

```
target:~$ ip address add 192.168.3.11/24 dev eth0
```

7.4 7.4 SD card

The i.MX 8M Plus supports a slot for Secure Digital cards to be used as general-purpose block devices. These devices can be used in the same way as any other block device.

Warning

These kinds of devices are hot-pluggable. Nevertheless, you must ensure not to unplug the device while it is still mounted. This may result in data loss!

After inserting an SD card, the kernel will generate new device nodes in /dev. The full device can be reached via its /dev/mmcblk1 device node. SD card partitions will show up as:

```
/dev/mmcblk1p<Y>
```

<Y> counts as the partition number starting from 1 to the max count of partitions on this device. The partitions can be formatted with any kind of file system and also handled in a standard manner, e.g. the mount and umount command work as expected.

Tip

These partition device nodes will only be available if the card contains a valid partition table (”hard disk” like handling). If no partition table is present, the whole device can be used as a file system (”floppy” like handling). In this case, /dev/mmcblk1 must be used for formatting and mounting. The cards are always mounted as being writable.

DT configuration for the MMC (SD card slot) interface can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L261

DT configuration for the eMMC interface can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L181

7.5 7.5 e.MMC Devices

PHYTEC modules like phyCORE-i.MX 8M Plus are populated with an e.MMC memory chip as the main storage. e.MMC devices contain raw Multi-Level Cells (MLC) or Triple-Level Cells (TLC) combined with a memory controller that handles ECC and wear leveling. They are connected via an SD/MMC interface to the i.MX 8M Plus and are represented as block devices in the Linux kernel like SD cards, flash drives, or hard disks.

The electric and protocol specifications are provided by JEDEC (https://www.jedec.org/standards-documents/technology-focus-areas/flash-memory-ssds-ufs-emmc/e-mmc). The e.MMC manufacturer’s datasheet is relatively short and meant to be read together with the supported version of the JEDEC e.MMC standard.

PHYTEC currently utilizes the e.MMC chips with JEDEC Version 5.0 and 5.1

7.5.1 7.5.1 Extended CSD Register

e.MMC devices have an extensive amount of extra information and settings that are available via the Extended CSD registers. For a detailed list of the registers, see manufacturer datasheets and the JEDEC standard.

In the Linux user space, you can query the registers:

```
target:~$ mmc extcsd read /dev/mmcblk2
```

You will see:

```
============================================
   Extended CSD rev 1.7 (MMC 5.0)
============================================

 Card Supported Command sets [S_CMD_SET: 0x01]
 [...]
```

7.5.2 7.5.2 Enabling Background Operations (BKOPS)

In contrast to raw NAND Flash, an e.MMC device contains a Flash Transfer Layer (FTL) that handles the wear leveling, block management, and ECC of the raw MLC or TLC. This requires some maintenance tasks (for example erasing unused blocks) that are performed regularly. These tasks are called **Background Operations (BKOPS)**.

By default (depending on the chip), the background operations may or may not be executed periodically, impacting the worst-case read and write latency.

The JEDEC Standard has specified a method since version v4.41 that the host can issue BKOPS manually. See the JEDEC Standard chapter Background Operations and the description of registers BKOPS_EN (Reg: 163) and BKOPS_START (Reg: 164) in the e.MMC datasheet for more details.

Meaning of Register BKOPS_EN (Reg: 163) Bit MANUAL_EN (Bit 0):

- Value 0: The host does not support the manual trigger of BKOPS. Device write performance suffers.
- Value 1: The host does support the manual trigger of BKOPS. It will issue BKOPS from time to time when it does not need the device.

The mechanism to issue background operations has been implemented in the Linux kernel since v3.7. You only have to enable BKOPS_EN on the e.MMC device (see below for details).

The JEDEC standard v5.1 introduces a new automatic BKOPS feature. It frees the host to trigger the background operations regularly because the device starts BKOPS itself when it is idle (see the description of bit AUTO_EN in register BKOPS_EN (Reg: 163)).

- To check whether *BKOPS_EN* is set, execute:

```
target:~$ mmc extcsd read /dev/mmcblk2 | grep BKOPS_EN
```

  The output will be, for example:

```
Enable background operations handshake [BKOPS_EN]: 0x01
#OR
Enable background operations handshake [BKOPS_EN]: 0x00
```

  Where value 0x00 means BKOPS_EN is disabled and device write performance suffers. Where value 0x01 means BKOPS_EN is enabled and the host will issue background operations from time to time.

- Enabling can be done with this command:

```
target:~$ target:~$ mmc --help

[...]
mmc bkops_en <auto|manual> <device>
    Enable the eMMC BKOPS feature on <device>.
    The auto (AUTO_EN) setting is only supported on eMMC 5.0 or newer.
    Setting auto won't have any effect if manual is set.
    NOTE!  Setting manual (MANUAL_EN) is one-time programmable (unreversible) change.
```

- To set the BKOPS_EN bit, execute:

```
target:~$ mmc bkops_en manual /dev/mmcblk2
```

- To ensure that the new setting is taken over and the kernel triggers BKOPS by itself, shut down the system:

```
target:~$ poweroff
```

Tip

The BKOPS_EN bit is one-time programmable only. It cannot be reversed.

7.5.3 7.5.3 Reliable Write

There are two different Reliable Write options:

1. Reliable Write option for a whole e.MMC device/partition.
2. Reliable Write for single write transactions.

Tip

Do not confuse e.MMC partitions with partitions of a DOS, MBR, or GPT partition table (see the previous section).

The first Reliable Write option is mostly already enabled on the e.MMCs mounted on the phyCORE-i.MX 8M Plus SoMs. To check this on the running target:

```
target:~$ mmc extcsd read /dev/mmcblk2 | grep -A 5 WR_REL_SET
Write reliability setting register [WR_REL_SET]: 0x1f
 user area: the device protects existing data if a power failure occurs during a write o
peration
 partition 1: the device protects existing data if a power failure occurs during a write
 operation
 partition 2: the device protects existing data if a power failure occurs during a write
 operation
 partition 3: the device protects existing data if a power failure occurs during a write
 operation
 partition 4: the device protects existing data if a power failure occurs during a write
 operation
--
 Device supports writing EXT_CSD_WR_REL_SET
 Device supports the enhanced def. of reliable write
```

Otherwise, it can be enabled with the mmc tool:

```
target:~$ mmc --help

[...]
mmc write_reliability set <-y|-n|-c> <partition> <device>
    Enable write reliability per partition for the <device>.
    Dry-run only unless -y or -c is passed.
    Use -c if more partitioning settings are still to come.
    NOTE!  This is a one-time programmable (unreversible) change.
```

The second Reliable Write option is the configuration bit Reliable Write Request parameter (bit 31) in command CMD23. It has been used in the kernel since v3.0 by file systems, e.g. ext4 for the journal and user space applications such as fdisk for the partition table. In the Linux kernel source code, it is handled via the flag REQ_META.

Conclusion: ext4 file system with mount option data=journal should be safe against power cuts. The file system check can recover the file system after a power failure, but data that was written just before the power cut may be lost. In any case, a consistent state of the file system can be recovered. To ensure data consistency for the files of an application, the system functions fdatasync or fsync should be used in the application.

7.5.4 7.5.4 Resizing ext4 Root Filesystem

When flashing the SD card image to e.MMC the ext4 root partition is not extended to the end of the e.MMC. parted can be used to expand the root partition. The example works for any block device such as e.MMC, SD card, or hard disk.

- Get the current device size:

```
target:~$ parted /dev/mmcblk2 print
```

- The output looks like this:

```
Model: MMC Q2J55L (sd/mmc)
Disk /dev/mmcblk2: 7617MB
Sect[ 1799.850385]  mmcblk2: p1 p2
or size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      4194kB  72.4MB  68.2MB  primary  fat16        boot, lba
 2      72.4MB  537MB   465MB   primary  ext4
```

- Use parted to resize the root partition to the max size of the device:

```
target:~$ parted /dev/mmcblk2 resizepart 2 100%
Information: You may need to update /etc/fstab.

target:~$ parted /dev/mmcblk2 print
Model: MMC Q2J55L (sd/mmc)
Disk /dev/mmcblk2: 7617MB
Sector size (logical/physical): 512[ 1974.191657]  mmcblk2: p1 p2
B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      4194kB  72.4MB  68.2MB  primary  fat16        boot, lba
 2      72.4MB  7617MB  7545MB  primary  ext4
```

- Resize the filesystem to a new partition size:

```
target:~$ resize2fs /dev/mmcblk2p2
resize2fs 1.46.1 (9-Feb-2021)
Filesystem at /dev/mmcblk2p2 is mounted on /; on-line resizing required
[ 131.609512] EXT4-fs (mmcblk2p2): resizing filesystem
from 454136 to 7367680 blocks
old_desc_blocks = 4, new_desc_blocks = 57
[ 131.970278] EXT4-fs (mmcblk2p2): resized filesystem to 7367680
The filesystem on /dev/mmcblk2p2 is now 7367680 (1k) blocks long
```

Increasing the filesystem size can be done while it is mounted. But you can also boot the board from an SD card and then resize the file system on the e.MMC partition while it is not mounted.

7.5.5 7.5.5 Enable pseudo-SLC Mode

e.MMC devices use MLC or TLC (https://en.wikipedia.org/wiki/Multi-level_cell) to store the data. Compared with SLC used in NAND Flash, MLC or TLC have lower reliability and a higher error rate at lower costs.

If you prefer reliability over storage capacity, you can enable the pseudo-SLC mode or SLC mode. The method used here employs the enhanced attribute, described in the JEDEC standard, which can be set for continuous regions of the device. The JEDEC standard does not specify the implementation details and the guarantees of the enhanced attribute. This is left to the chipmaker. For the Micron chips, the enhanced attribute increases the reliability but also halves the capacity.

Warning

When enabling the enhanced attribute on the device, all data will be lost.

The following sequence shows how to enable the enhanced attribute.

- First obtain the current size of the e.MMC device with:

```
target:~$ parted -m /dev/mmcblk2 unit B print
```

  You will receive:

```
BYT;
/dev/mmcblk2:63652757504B:sd/mmc:512:512:unknown:MMC S0J58X:;
```

  As you can see this device has 63652757504 Byte = 60704 MiB.

- To get the maximum size of the device after pseudo-SLC is enabled use:

```
target:~$ mmc extcsd read /dev/mmcblk2 | grep ENH_SIZE_MULT -A 1
```

  which shows, for example:

```
Max Enhanced Area Size [MAX_ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
--
Enhanced User Data Area Size [ENH_SIZE_MULT]: 0x000000
i.e. 0 KiB
```

  Here the maximum size is 3719168 KiB = 3632 MiB.

- Now, you can set enhanced attribute for the whole device, e.g. 3719168 KiB, by typing:

```
target:~$ mmc enh_area set -y 0 3719168 /dev/mmcblk2
```

  You will get:

```
Done setting ENH_USR area on /dev/mmcblk2
setting OTP PARTITION_SETTING_COMPLETED!
Setting OTP PARTITION_SETTING_COMPLETED on /dev/mmcblk2 SUCCESS
Device power cycle needed for settings to take effect.
Confirm that PARTITION_SETTING_COMPLETED bit is set using 'extcsd read' after power cycle
```

- To ensure that the new setting has taken over, shut down the system:

```
target:~$ poweroff
```

and perform a power cycle. It is recommended that you verify the settings now.

- First, check the value of ENH_SIZE_MULT which must be 3719168 KiB:

```
target:~$ mmc extcsd read /dev/mmcblk2 | grep ENH_SIZE_MULT  -A 1
```

You should receive:

```
Max Enhanced Area Size [MAX_ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
--
Enhanced User Data Area Size [ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
```

- Finally, check the size of the device:

```
target:~$ parted -m /dev/mmcblk2 unit B print
BYT;
/dev/mmcblk2:31742492672B:sd/mmc:512:512:unknown:MMC S0J58X:;
```

7.5.6 7.5.6 Erasing the Device

It is possible to erase the e.MMC device directly rather than overwriting it with zeros. The e.MMC block management algorithm will erase the underlying MLC or TLC or mark these blocks as discard. The data on the device is lost and will be read back as zeros.

- After booting from SD card execute:

```
target:~$ blkdiscard -f --secure /dev/mmcblk2
```

The option –secure ensures that the command waits until the eMMC device has erased all blocks. The -f (force) option disables all checking before erasing and it is needed when the eMMC device contains existing partitions with data.

Tip

```
target:~$ dd if=/dev/zero of=/dev/mmcblk2 conv=fsync
```

also destroys all information on the device, but this command is bad for wear leveling and takes much longer!

7.5.7 7.5.7 e.MMC Boot Partitions

An e.MMC device contains four different hardware partitions: user, boot1, boot2, and rpmb.

The user partition is called the User Data Area in the JEDEC standard and is the main storage partition. The partitions boot1 and boot2 can be used to host the bootloader and are more reliable. Which partition the i.MX 8M Plus uses to load the bootloader is controlled by the boot configuration of the e.MMC device. The partition rpmb is a small partition and can only be accessed via a trusted mechanism.

Furthermore, the user partition can be divided into four user-defined General Purpose Area Partitions. An explanation of this feature exceeds the scope of this document. For further information, see the JEDEC Standard Chapter 7.2 Partition Management.

Tip

Do not confuse e.MMC partitions with partitions of a DOS, MBR, or GPT partition table.

The current PHYTEC BSP does not use the extra partitioning feature of e.MMC devices. The U-Boot is flashed at the beginning of the user partition. The U-Boot environment is placed at a fixed location after the U-Boot. An MBR partition table is used to create two partitions, a FAT32 boot, and ext4 rootfs partition. They are located right after the U-Boot and the U-Boot environment. The FAT32 boot partition contains the kernel and device tree.

With e.MMC flash storage it is possible to use the dedicated boot partitions for redundantly storing the bootloader. The Bootloader environment still resides in the user area before the first partition. The user area also still contains the bootloader which the image first shipped during its initialization process. Below is an example, to flash the bootloader to one of the two boot partitions and switch the boot device via userspace commands.

7.5.7.1 Via userspace Commands

On the host, run:

```
host:~$ scp <bootloader> root@192.168.3.11:/tmp/
```

The partitions boot1 and boot2 are read-only by default. To write to them from user space, you have to disable `force_ro` in the sysfs.

To manually write the bootloader to the e.MMC boot partitions, first disable the write protection:

```
target:~$ echo 0 > /sys/block/mmcblk2boot0/force_ro
target:~$ echo 0 > /sys/block/mmcblk2boot1/force_ro
```

Write the bootloader to the e.MMC boot partitions:

```
target:~$ dd if=/tmp/<bootloader> of=/dev/mmcblk2boot0
target:~$ dd if=/tmp/<bootloader> of=/dev/mmcblk2boot1
```

The following table is for the offset of the i.MX 8M Plus SoC:

SoC	Offset User Area	Offset Boot Partition	e.MMC Device
i.MX 8M Plus	32 kiB	0 kiB	/dev/mmcblk2

After that set the boot partition from user space using the `mmc` tool:

(for 'boot0') :

```
target:~$ mmc bootpart enable 1 0 /dev/mmcblk2
```

(for 'boot1') :

```
target:~$ mmc bootpart enable 2 0 /dev/mmcblk2
```

To disable booting from the e.MMC boot partitions simply enter the following command:

```
target:~$ mmc bootpart enable 0 0 /dev/mmcblk2
```

To explicitly enable booting from the e.MMC user area, run:

```
target:~$ mmc bootpart enable 7 0 /dev/mmcblk2
```

7.5.7.2 Automatic failover

The ROM loader implements an automatic failover mechanism for e.MMC boot partitions. If booting from the primary partition fails, the system automatically attempts to boot from the secondary partition. This failover is indicated by a change in the boot message from `Boot Stage: Primary boot` to `Boot Stage: Secondary boot`. This functionality is limited to boot0 and boot1 partitions and does not apply to the user area.

7.6 7.6 SPI Master

The i.MX 8M Plus controller has a FlexSPI and an ECSPI IP core included. The FlexSPI host controller supports two SPI channels with up to 4 devices. Each channel supports Single/Dual/Quad/Octal mode data transfer (1/2/4/8 bidirectional data lines). The ECSPI controller supports 3 SPI interfaces with one dedicated chip selected for each interface. As chip selects should be realized with GPIOs, more than one device on each channel is possible.

The definition of the SPI master node in the device tree can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L67

7.7 7.7 GPIOs

The phyBOARD-Pollux has a set of pins especially dedicated to user I/Os. Those pins are connected directly to i.MX 8M Plus pins and are muxed as GPIOs. They are directly usable in Linux userspace. The processor has organized its GPIOs into five banks of 32 GPIOs each (GPIO1 – GPIO5) GPIOs. gpiochip0, gpiochip32, gpiochip64, gpiochip96, and gpiochip128 are the sysfs representation of these internal i.MX 8M Plus GPIO banks GPIO1 – GPIO5.

The GPIOs are identified as GPIO<X>_<Y> (e.g. GPIO5_07). <X> identifies the GPIO bank and counts from 1 to 5, while <Y> stands for the GPIO within the bank. <Y> is being counted from 0 to 31 (32 GPIOs on each bank).

By contrast, the Linux kernel uses a single integer to enumerate all available GPIOs in the system. The formula to calculate the right number is:

```
Linux GPIO number: <N> = (<X> - 1) * 32 + <Y>
```

Accessing GPIOs from userspace will be done using the libgpiod. It provides a library and tools for interacting with the Linux GPIO character device. Examples of some usages of various tools:

- Detecting the gpiochips on the chip:

  ```
  target:~$ gpiodetect
  gpiochip0 [30200000.gpio] (32 lines)
  gpiochip1 [30210000.gpio] (32 lines)
  gpiochip2 [30220000.gpio] (32 lines)
  gpiochip3 [30230000.gpio] (32 lines)
  gpiochip4 [30240000.gpio] (32 lines)
  ```

- Show detailed information about the gpiochips. Like their names, consumers, direction, active state, and additional flags:

```
target:~$ gpioinfo -c gpiochip0
```

- Read the value of a GPIO (e.g GPIO 20 from chip0):

```
target:~$ gpioget -c gpiochip0 20
```

- Set the value of GPIO 20 on chip0 to 0:

```
target:~$ gpioset -c gpiochip0 20=0
```

The GPIO is only set as long as the program is running. By default the program is running until the user exits with `CTRL+C` to the console. With the `-z` option, the program will return to the console and keep running in the background. It will keep blocking the resource until it is terminated. This can be done with:

```
target:~$ ps | grep "gpioset"
1116 root      0:00 gpioset -z -c gpiochip0 20 0
1120 root      0:00 grep gpioset
target:~$ kill 1116
```

- Help text of gpioset shows possible options:

```
target:~$ gpioset --help
Usage: gpioset [OPTIONS] <line=value>...

Set values of GPIO lines.

Lines are specified by name, or optionally by offset if the chip option
is provided.
Values may be '1' or '0', or equivalently 'active'/'inactive' or 'on'/'off'.

The line output state is maintained until the process exits, but after that
is not guaranteed.

Options:
      --banner            display a banner on successful startup
  -b, --bias <bias>       specify the line bias
                  Possible values: 'pull-down', 'pull-up', 'disabled'.
                  (default is to leave bias unchanged)
      --by-name           treat lines as names even if they would parse as an offset
  -c, --chip <chip>       restrict scope to a particular chip
  -C, --consumer <name>   consumer name applied to requested lines (default is 'gpioset')
  -d, --drive <drive>     specify the line drive mode
                  Possible values: 'push-pull', 'open-drain', 'open-source'.
                  (default is 'push-pull')
  -h, --help              display this help and exit
  -l, --active-low        treat the line as active low
  -p, --hold-period <period>
                  the minimum time period to hold lines at the requested values
  -s, --strict            abort if requested line names are not unique
  -t, --toggle <period>[,period]...
                  toggle the line(s) after the specified period(s)
                  If the last period is non-zero then the sequence repeats.
```

(continues on next page)

(continued from previous page)

```
      --unquoted  don't quote line names
  -v, --version           output version information and exit
  -z, --daemonize set values then detach from the controlling terminal

Chips:
    A GPIO chip may be identified by number, name, or path.
    e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

Periods:
    Periods are taken as milliseconds unless units are specified. e.g. 10us.
    Supported units are 's', 'ms', and 'us'.

*Note*
    The state of a GPIO line controlled over the character device reverts to default
    when the last process referencing the file descriptor representing the device file␣
↪exits.
    This means that it's wrong to run gpioset, have it exit and expect the line to continue
    being driven high or low. It may happen if given pin is floating but it must be␣
↪interpreted
    as undefined behavior.
```

Warning

Some of the user IOs are used for special functions. Before using a user IO, refer to the schematic or the hardware manual of your board to ensure that it is not already in use.

7.7.1 7.7.1 GPIOs via sysfs

Warning

Accessing gpios via sysfs is deprecated and we encourage to use libgpiod instead.

Support to access GPIOs via sysfs is not enabled by default any more. It is only possible with manually enabling `CONFIG_GPIO_SYSFS` in the kernel configuration. To make `CONFIG_GPIO_SYSFS` visible in menuconfig the option `CONFIG_EXPERT` has to be enabled first.

You can also add this option for example to the defconfig you use in `arch/arm64/configs/` in the linux kernel sources. For our NXP based releases, this could be for example defconfig:

```
..
CONFIG_EXPERT=y
CONFIG_GPIO_SYSFS=y
..
```

Otherwise you can create a new config fragment. This is described in our Yocto Reference Manual.

7.8 7.8 LEDs

If any LEDs are connected to GPIOs, you have the possibility to access them by a special LED driver interface instead of the general GPIO interface (section GPIOs). You will then access them using `/sys/class/leds/` instead of `/sys/class/gpio/`. The maximum brightness of the LEDs can be read from the `max_brightness` file. The brightness file will set the brightness of the LED (taking a value from 0 up to max_brightness). Most LEDs do not have hardware brightness support and will just be turned on by all non-zero brightness settings.

Below is a simple example.

To get all available LEDs, type:

```
target:~$ ls /sys/class/leds
led-1@  led-2@  led-3@  mmc1::@  mmc2::@
```

The phyBOARD-Pollux provides the following LED indicators: led-0, led-1 and led-2.

- To toggle the LEDs ON:

  ```
  target:~$ echo 255 > /sys/class/leds/led-1/brightness
  ```

- To toggle OFF:

  ```
  target:~$ echo 0 > /sys/class/leds/led-1/brightness
  ```

Device tree configuration for the User I/O configuration can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L160

7.9 7.9 I²C Bus

The i.MX 8M Plus contains several Multimaster fast-mode I²C modules. PHYTEC boards provide plenty of different I²C devices connected to the I²C modules of the i.MX 8M Plus. This section describes the basic device usage and its DT representation of some I²C devices integrated into our phyBOARD-Pollux.

The device tree node for i2c contains settings such as clock-frequency to set the bus frequency and the pin control settings including scl-gpios and sda-gpios which are alternate pin configurations used for bus recovery.

General I²C1 bus configuration (e.g. imx8mp-phycore-som.dtsi): https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L81

General I²C2 bus configuration (e.g. imx8mp-phyboard-pollux-rdk.dts) https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L145

7.10 7.10 EEPROM

On the phyCORE-i.MX 8M Plus there is an i2c EEPROM flash populated. It has two addresses. The main EEPROM space (bus: I2C-0 address: 0x51) can be accessed via the sysfs interface in Linux. The first 256 bytes of the main EEPROM and the ID-page (bus: I2C-0 address: 0x59) are used for board detection and must not be overwritten. Therefore the ID-page can not be accessed via the sysfs interface. Overwriting reserved spaces will result in boot issues.

Note

If you deleted reserved EEPROM spaces, please contact our support!

7.10.1 7.10.1 I2C EEPROM on phyCORE-i.MX 8M Plus

Warning

The EEPROM ID page (bus: I2C-0 addr: 0x59) and the first 256 bytes of the normal EEPROM area (bus: I2C-0 addr: 0x51) should not be erased or overwritten. As this will influence the behavior of the bootloader. The board might not boot correctly anymore.

The I2C EEPROM on the phyCORE-i.MX 8M Plus SoM is connected to I2C address 0x51 on the I2C-0 bus. It is possible to read and write directly to the device populated:

```
target:~$ hexdump -c /sys/class/i2c-dev/i2c-0/device/0-0051/eeprom
```

To read and print the first 1024 bytes of the EEPROM as a hex number, execute:

```
target:~$ dd if=/sys/class/i2c-dev/i2c-0/device/0-0051/eeprom bs=1 count=1024  | od -x
```

To fill the 4KiB EEPROM (bus: I2C-0 addr: 0x51) with zeros leaving out the EEPROM data use:

```
target:~$ dd if=/dev/zero of=/sys/class/i2c-dev/i2c-0/device/0-0051/eeprom seek=1 bs=256 count=15
```

7.10.2 7.10.2 EEPROM SoM Detection

The I2C EEPROM, populated on the phyCORE-i.MX 8M Plus, has a separate ID page that is addressable over I2C address 0x59 on bus 0 and a normal area that is addressable over I2C address 0x51 on bus 0. PHYTEC uses this data area of 32 Bytes to store information about the SoM. This includes PCB revision and mounting options.

The EEPROM data is read at a really early stage during startup. It is used to select the correct RAM configuration. This makes it possible to use the same bootloader image for different RAM sizes and choose the correct DTS overlays automatically.

If the EEPROM ID page data and the first 256 bytes of the normal area are deleted, the bootloader will fall back to the phyCORE-i.MX 8M Plus Kit RAM setup, which is 2GiB RAM.

Warning

The EEPROM ID page (bus: I2C-0 addr: 0x59) and the first 256 bytes of the normal EEPROM area (bus: I2C-0 addr: 0x51) should not be erased or overwritten. As this will influence the behavior of the bootloader. The board might not boot correctly anymore.

SoMs that are flashed with data format API revision 2 will print out information about the module in the early stage.

DT representation, e.g. in phyCORE-i.MX 8M Plus file imx8mp-phycore-som.dtsi can be found in our PHYTEC git: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L169

7.11 7.11 RTC

RTCs can be accessed via `/dev/rtc*`. Because PHYTEC boards have often more than one RTC, there might be more than one RTC device file.

- To find the name of the RTC device, you can read its sysfs entry with:

```
target:~$ cat /sys/class/rtc/rtc*/name
```

- You will get, for example:

```
rtc-rv3028 0-0052
snvs_rtc 30370000.snvs:snvs-rtc-lp
```

Tip

This will list all RTCs including the non-I²C RTCs. Linux assigns RTC device IDs based on the device tree/aliases entries if present.

Date and time can be manipulated with the `hwclock` tool and the date command. To show the current date and time set on the target:

```
target:~$ date
Thu Jan  1 00:01:26 UTC 1970
```

Change the date and time with the date command. The date command sets the time with the following syntax "YYYY-MM-DD hh:mm:ss (+|-)hh:mm":

```
target:~$ date -s "2022-03-02 11:15:00 +0100"
Wed Mar  2 10:15:00 UTC 2022
```

Note

Your timezone (in this example +0100) may vary.

Using the date command does not change the time and date of the RTC, so if we were to restart the target those changes would be discarded. To write to the RTC we need to use the `hwclock` command. Write the current date and time (set with the date command) to the RTC using the `hwclock` tool and reboot the target to check if the changes were applied to the RTC:

```
target:~$ hwclock -w
target:~$ reboot
    .
    .
    .
target:~$ date
Wed Mar  2 10:34:06 UTC 2022
```

To set the time and date from the RTC use:

```
target:~$ date
Thu Jan  1 01:00:02 UTC 1970
target:~$ hwclock -s
target:~$ date
Wed Mar  2 10:45:01 UTC 2022
```

7.11.1 7.11.1 RTC Parameters

RTCs have a few abilities which can be read/set with the help of `hwclock` tool.

- We can check RTC supported features with:

```
target:~$ hwclock --param-get features
The RTC parameter 0x0 is set to 0x71.
```

  What this value means is encoded in kernel, each set bit translates to:

```
#define RTC_FEATURE_ALARM               0
#define RTC_FEATURE_ALARM_RES_MINUTE    1
#define RTC_FEATURE_NEED_WEEK_DAY       2
#define RTC_FEATURE_ALARM_RES_2S        3
#define RTC_FEATURE_UPDATE_INTERRUPT    4
#define RTC_FEATURE_CORRECTION          5
#define RTC_FEATURE_BACKUP_SWITCH_MODE  6
#define RTC_FEATURE_ALARM_WAKEUP_ONLY   7
#define RTC_FEATURE_CNT                 8
```

- We can check RTC BSM (Backup Switchover Mode) with:

```
target:~$ hwclock --param-get bsm
The RTC parameter 0x2 is set to 0x1.
```

- We can set RTC BSM with:

```
target:~$ hwclock --param-set bsm=0x2
The RTC parameter 0x2 will be set to 0x2.
```

  What BSM values mean translates to these values:

```
#define RTC_BSM_DISABLED   0
#define RTC_BSM_DIRECT     1
#define RTC_BSM_LEVEL      2
#define RTC_BSM_STANDBY    3
```

Tip

You should set BSM mode to DSM or LSM for RTC to switch to backup power source when the initial power source is not available. Check **RV-3028** RTC datasheet to read what LSM (Level Switching Mode) and DSM (Direct Switching Mode) actually mean.

DT representation for I²C RTCs: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phycore-som.dtsi#L175

7.12 7.12 USB Host Controller

The USB controller of the i.MX 8M Plus SoC provides a low-cost connectivity solution for numerous consumer portable devices by providing a mechanism for data transfer between USB devices with a line/bus speed of up to 4 Gbit/s (SuperSpeed 'SS'). The USB subsystem has two independent USB controller cores. Both cores are capable of acting as a USB peripheral device or a USB host. Each is connected to a USB 3.0 PHY.

The unified BSP includes support for mass storage devices and keyboards. Other USB-related device drivers must be enabled in the kernel configuration on demand. Due to udev, all mass storage devices connected get unique IDs and can be found in `/dev/disk/by-id`. These IDs can be used in `/etc/fstab` to mount the different USB memory devices in different ways.

In U-Boot USB mass storage devices will not automatically be detected on insertion. Instead USB devices will be probed once when USB is started.

```
u-boot=> usb start
```

If USB is already started and a new device is plugged in, USB needs to be restarted to trigger the device detection again.

```
u-boot=> usb reset
```

DT representation for USB Host: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L220

7.13 7.13 CAN FD

The phyBOARD-Pollux has two flexCAN interfaces supporting CAN FD. They are supported by the Linux standard CAN framework which builds upon then the Linux network layer. Using this framework, the CAN interfaces behave like an ordinary Linux network device, with some additional features special to CAN. More information can be found in the Linux Kernel documentation: https://www.kernel.org/doc/html/latest/networking/can.html

- Use:

  ```
  target:~$ ip link
  ```

  to see the state of the interfaces.

- To get information on can0, such as bit rate and error counters, type:

  ```
  target:~$ ip -d -s link show can0
  4: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group␣
  ↪default qlen 10
     link/can  promiscuity 0 allmulti 0 minmtu 0 maxmtu 0
     can <FD> state ERROR-ACTIVE (berr-counter tx 0 rx 0) restart-ms 0
           bitrate 500000 sample-point 0.875
           tq 25 prop-seg 37 phase-seg1 32 phase-seg2 10 sjw 5 brp 1
           flexcan: tseg1 2..96 tseg2 2..32 sjw 1..16 brp 1..1024 brp_inc 1
           dbitrate 2000000 dsample-point 0.750
           dtq 25 dprop-seg 7 dphase-seg1 7 dphase-seg2 5 dsjw 2 dbrp 1
           flexcan: dtseg1 2..39 dtseg2 2..8 dsjw 1..4 dbrp 1..1024 dbrp_inc 1
           clock 40000000
           re-started bus-errors arbit-lost error-warn error-pass bus-off
           0          0          0          0          0          0         numtxqueues 1␣
  ↪numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535␣
  ↪gro_max_size 65536
  gso_ipv4_max_size 65536 gro_ipv4_max_size 65536 parentbus platform parentdev 443a0000.can
     RX:  bytes packets errors dropped  missed   mcast
               0       0      0       0       0       0
     TX:  bytes packets errors dropped carrier collsns
               0       0      0       0       0       0
  ```

The output contains a standard set of parameters also shown for Ethernet interfaces, so not all of these are necessarily relevant for CAN (for example the MAC address). The following output parameters contain useful information:

can0	Interface Name
NOARP	CAN cannot use ARP protocol
MTU	Maximum Transfer Unit
RX packets	Number of Received Packets
TX packets	Number of Transmitted Packets
RX bytes	Number of Received Bytes
TX bytes	Number of Transmitted Bytes
errors…	Bus Error Statistics

The CAN configuration is done in the systemd configuration file `/lib/systemd/network/can0.network`. For a persistent change of (as an example, the default bitrates), change the configuration in the BSP under `./meta-ampliphy/recipes-core/systemd/systemd-conf/can0.network` in the root filesystem and rebuild the root filesystem.

Note

By default, we enable CAN-FD (flexible datarate) in our BSP. In case CAN Classic is required one needs to remove options `FDMode` and `DataBitRate` from the `/lib/systemd/network/can0.network` file.

To disable flexible datarate manually, one can use:

```
target:~$ ip link set can0 down
target:~$ ip link set can0 type can bitrate 500000 dbitrate 0 fd off
target:~$ ip link set can0 up
```

The bitrate can also be changed manually, for example:

```
target:~$ ip link set can0 down
target:~$ ip link set can0 txqueuelen 10 up type can bitrate 500000 sample-point 0.75 dbitrate␣
↪4000000 dsample-point 0.8 fd on
target:~$ ip link set can0 up
```

You can send messages with cansend or receive messages with candump:

```
target:~$ cansend can0 123#45.67
target:~$ candump can0
```

To generate random CAN traffic for testing purposes, use cangen:

```
target:~$ cangen
```

`cansend --help` and `candump --help` provide help messages for further information on options and usage.

Device Tree CAN configuration of imx8mp-phyboard-pollux-rdk.dts: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L130

7.14 7.14 Video

7.14.1 7.14.1 Videos with Gstreamer

One example video is installed by default in the BSP at */usr/share/qtphy/videos/*. Start the video playback with one of these commands:

```
target:~$ gst-launch-1.0 -v filesrc location=/usr/share/qtphy/videos/caminandes_3_llamigos_720p_
↪vp9.webm ! decodebin name=decoder decoder. ! videoconvert ! waylandsink fullscreen=true
```

- Or:

```
target:~$ gst-play-1.0 /usr/share/qtphy/videos/caminandes_3_llamigos_720p_vp9.webm --videosink␣
↪waylandsink
```

Note

The mainline BSP currently only supports software rendering.

7.15 7.15 Display

The phyBOARD-Pollux supports LVDS output via the LVDS1 connector on the carrier board. The LVDS interface is enabled by default.

7.15.1 7.15.1 Weston Configuration

Weston will work without any additional configuration. Configuration options are done at `/etc/xdg/weston/weston.ini`.

Device tree description of LVDS can be found here: https://github.com/phytec/linux-phytec/blob/v6.6.21-phy1/arch/arm64/boot/dts/freescale/imx8mp-phyboard-pollux-rdk.dts#L182

7.15.2 7.15.2 Qt Demo

With the `phytec-qt6demo-image`, Weston starts during boot. Our Qt6 demo application named "qtphy" can be stopped with:

```
target:~$ systemctl stop qtphy
```

- To start the demo again, run:

  ```
  target:~$ systemctl start qtphy
  ```

- To disable autostart of the demo, run:

  ```
  target:~$ systemctl disable qtphy
  ```

- To enable autostart of the demo, run:

  ```
  target:~$ systemctl enable qtphy
  ```

- Weston can be stopped with:

```
target:~$ systemctl stop weston
```

Note

The Qt demo must be closed before Weston can be closed.

7.15.3 7.15.3 Backlight Control

If a display is connected to the PHYTEC board, you can control its backlight with the Linux kernel sysfs interface. All available backlight devices in the system can be found in the folder /sys/class/backlight. Reading the appropriate files and writing to them allows you to control the backlight.

Note

Some boards with multiple display connectors might have multiple backlight controls in /sys/class/backlight. For example: backlight0 and backlight1

- To get, for example, the maximum brightness level (max_brightness) execute:

  ```
  target:~$ cat /sys/class/backlight/backlight/max_brightness
  ```

  Valid brightness values are 0 to <max_brightness>.

- To obtain the current brightness level, type:

  ```
  target:~$ cat /sys/class/backlight/backlight/brightness
  ```

- Write to the file brightness to change the brightness:

  ```
  target:~$ echo 0 > /sys/class/backlight/backlight/brightness
  ```

  turns the backlight off for example.

  For documentation of all files, see https://www.kernel.org/doc/Documentation/ABI/stable/sysfs-class-backlight.

Note

We noticed some visible backlight flickering on brightness level 1 (probably due to frequency problems with the hardware).

7.16 7.16 Power Management

7.16.1 7.16.1 CPU Core Frequency Scaling

The CPU in the i.MX 8M Plus SoC is able to scale the clock frequency and the voltage. This is used to save power when the full performance of the CPU is not needed. Scaling the frequency and the voltage is referred to as 'Dynamic Voltage and Frequency Scaling' (DVFS). The i.MX 8M Plus BSP supports the DVFS feature. The Linux kernel provides a DVFS framework that allows each CPU core to have a min/max frequency and a governor that governs it. Depending on the i.MX 8 variant used, several different frequencies are supported.

Tip

Although the DVFS framework provides frequency settings for each CPU core, a change in the frequency settings of one CPU core always affects all other CPU cores too. So all CPU cores always share the same DVFS setting. An individual DVFS setting for each core is not possible.

- To get a complete list type:

```
target:~$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

  In case you have, for example, i.MX 8MPlus CPU with a maximum of approximately 1,6 GHz, the result will be:

```
1200000 1600000
```

- To ask for the current frequency type:

```
target:~$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
```

So-called governors are automatically selecting one of these frequencies in accordance with their goals.

- List all governors available with the following command:

```
target:~$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_governors
```

  The result will be:

```
ondemand userspace performance schedutil
```

- **ondemand** (default) switches between possible CPU core frequencies in reference to the current system load. When the system load increases above a specific limit, it increases the CPU core frequency immediately.
- **performance** always selects the highest possible CPU core frequency.
- **userspace** allows the user or userspace program running as root to set a specific frequency (e.g. to 1600000). Type:
- In order to ask for the current governor, type:

```
target:~$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

  You will normally get:

```
schedutil
```

- Switching over to another governor (e.g. userspace) is done with:

```
target:~$ echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

- Now you can set the speed:

```
target:~$ echo 1600000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

For more detailed information about the governors, refer to the Linux kernel documentation in the linux kernel repository at `Documentation/admin-guide/pm/cpufreq.rst`.

7.16.2 7.16.2 CPU Core Management

The i.MX 8M Plus SoC can have multiple processor cores on the die. The i.MX 8M Plus, for example, has 4 ARM Cores which can be turned on and off individually at runtime.

- To see all available cores in the system, execute:

```
target:~$ ls /sys/devices/system/cpu  -1
```

- This will show, for example:

```
cpu0    cpu1   cpu2   cpu3   cpufreq
[...]
```

  Here the system has four processor cores. By default, all available cores in the system are enabled to get maximum performance.

- To switch off a single-core, execute:

```
target:~$ echo 0 > /sys/devices/system/cpu/cpu3/online
```

  As confirmation, you will see:

```
[  110.505012] psci: CPU3 killed
```

  Now the core is powered down and no more processes are scheduled on this core.

- You can use top to see a graphical overview of the cores and processes:

```
target:~$ htop
```

- To power up the core again, execute:

```
target:~$ echo 1 > /sys/devices/system/cpu/cpu3/online
```

7.17 7.17 Thermal Management

7.17.1 7.17.1 U-Boot

The previous temperature control in the U-Boot was not satisfactory. Now the u-boot has a temperature shutdown to prevent the board from getting too hot during booting. The temperatures at which the shutdown occurs are identical to those in the kernel.

The individual temperature ranges with the current temperature are displayed in the boot log:

```
CPU:   Industrial temperature grade (-40C to 105C) at 33C
```

7.17.2 7.17.2 Kernel

The Linux kernel has integrated thermal management that is capable of monitoring SoC temperatures, reducing the CPU frequency, driving fans, advising other drivers to reduce the power consumption of devices, and – worst-case – shutting down the system gracefully (https://www.kernel.org/doc/Documentation/thermal/sysfs-api.txt).

This section describes how the thermal management kernel API is used for the i.MX 8M Plus SoC platform. The i.MX 8 has internal temperature sensors for the SoC.

- The current temperature can be read in millicelsius with:

```
target:~$ cat /sys/class/thermal/thermal_zone0/temp
```

- You will get, for example:

```
49000
```

There are two trip points registered by the imx_thermal kernel driver. These differ depending on the CPU variant. A distinction is made between Industrial and Commercial.

	Commercial	Industrial
passive (warning)	85°C	95°C
critical (shutdown)	90°C	100°C

(see kernel sysfs folder `/sys/class/thermal/thermal_zone0/`)

The kernel thermal management uses these trip points to trigger events and change the cooling behavior. The following thermal policies (also named thermal governors) are available in the kernel: Step Wise, Fair Share, Bang Bang, and Userspace. The default policy used in the BSP is step_wise. If the value of the SoC temperature in the sysfs file temp is above *trip_point_0*, the CPU frequency is set to the lowest CPU frequency. When the SoC temperature drops below *trip_point_0* again, the throttling is released.

Note

The actual values of the thermal trip points may differ since we mount CPUs with different temperature grades.

7.18 7.18 Watchdog

The PHYTEC i.MX 8M Plus modules include a hardware watchdog that is able to reset the board when the system hangs. The watchdog is started on default in U-Boot with a timeout of 60s. So even during early kernel start, the watchdog is already up and running. The Linux kernel driver takes control over the watchdog and makes sure that it is fed. This section explains how to configure the watchdog in Linux using systemd to check for system hangs and during reboot.

7.18.1 7.18.1 Watchdog Support in systemd

Systemd has included hardware watchdog support since version 183.

- To activate watchdog support, the file system.conf in `/etc/systemd/` has to be adapted by enabling the options:

```
RuntimeWatchdogSec=60s
RebootWatchdogSec=10min
```

RuntimeWatchdogSec defines the timeout value of the watchdog, while *RebootWatchdogSec* defines the timeout when the system is rebooted. For more detailed information about hardware watchdogs under systemd can be found at http://0pointer.de/blog/projects/watchdog.html. The changes will take effect after a reboot or run:

```
target:~$  systemctl daemon-reload
```

7.19 7.19 snvs Power Key

The X_ONOFF pin connected to the ON/OFF button can be pressed long to trigger Power OFF without SW intervention. With the *snvs_pwrkey* driver, the KEY_POWER event is also reported to userspace when the button is pressed. On default, systemd is configured to ignore such events. The function of Power OFF without SW intervention are not configured. Triggering a power off with systemd when pushing the ON/OFF button can be configured under `/etc/systemd/logind.conf` and set using:

```
HandlePowerKey=poweroff
```

7.20 7.20 On-Chip OTP Controller (OCOTP_CTRL) - eFuses

The i.MX 8M Plus provides one-time programmable fuses to store information such as the MAC address, boot configuration, and other permanent settings ("On-Chip OTP Controller (OCOTP_CTRL)" in the i.MX 8M Plus Reference Manual). The following list is an abstract from the i.MX 8M Plus Reference Manual and includes some useful registers in the OCOTP_CTRL (at base address 0x30350000):

Name	Bank	Word	Memory offset at 0x30350000	Description
OCOTP_MAC_	9	0	0x640	contains lower 32 bits of ENET0 MAC address
OCOTP_MAC_	9	1	0x650	contains upper 16 bits of ENET0 MAC address and the lower 16 bits of ENET1 MAC address
OCOTP_MAC_	9	2	0x660	contains upper 32 bits of ENET1 MAC address

A complete list and a detailed mapping between the fuses in the OCOTP_CTRL and the boot/mac/... configuration are available in the section "Fuse Map" of the i.MX 8M Plus Security Reference Manual.

7.20.1 7.20.1 Reading Fuse Values in uBoot

You can read the content of a fuse using memory-mapped shadow registers. To calculate the memory address, use the fuse Bank and Word in the following formula:

OCOTP_MAC_ADDR:

```
u-boot=> fuse read 9 0
```

7.20.2 7.20.2 Reading Fuse Values in Linux

To access the content of the fuses in Linux NXP provides the NVMEM_IMX_OCOTP module. All fuse content of the memory-mapped shadow registers is accessible via sysfs:

```
target:~$ hexdump /sys/devices/platform/soc@0/30000000.bus/30350000.efuse/imx-ocotp0/nvmem
```